

APPROVED V1.1

Production Architecture Contract

Micro-Gig Marketplace Platform

Project Requirements Document (PRD) — Comprehensive Functional, Technical, and Architectural Specification for an Escrow-Backed Micro-Freelance Monolith.

Service-as-a-Product (SaaP)

Fixed-price digital deliverables under \$50, delivered in 24-48 hours, eliminating proposals and scope bidding.

Double-Entry Escrow Ledger

Mathematical zero-sum ledger across 5 account kinds with PostgreSQL deferred constraint triggers.

Modular Monolith Architecture

Next.js 14 App Router with 8 decoupled domain modules, avoiding microservice network overhead.

Automated Time-Gated Resolution

72-hour auto-completion timers in BullMQ backed by a 10-minute resilient SQL reconciler cron.

VERSION

1.1 (Fiverr Aligned)

TARGET TIMELINE

6 Weeks (Team of 4)

ENGINEERING LEADS

Core Architecture
Team

EFFECTIVE DATE

September 30, 2026

Micro-Gig Marketplace Platform

Project Requirements Document (PRD)

Version: 1.1

Date: September 30, 2026

Status: Approved — Updated from Fiverr Benchmark & Security Audit

Prepared By: Core Engineering Team

Revision History: What changed in v1.1

What changed in v1.1: v1.0 was cross-checked against 48 Fiverr screenshots (buyer, seller, messaging, billing, analytics, onboarding). This version (1) adds benchmark-driven requirements, (2) fixes defects found in the v1.0 ledger, API and security design, and (3) re-scopes the sprint plan. Sections unchanged from v1.0 (full DDL, Prisma schema, Docker Compose, env vars, queue definitions) carry over as Appendices A-E of v1.0.

Version 1.0 was rigorously cross-checked against **48 real-world Fiverr production screenshots** covering the entire user journey: buyer search and checkout, seller onboarding and gig creation, order workspace tabs, async messaging, billing and wallet analytics, and administrative verification.

This version (v1.1) achieves three critical engineering objectives: 1. **Adds Benchmark-Driven Requirements:** Adopts essential marketplace primitives observed in production (a 5-step gig creation wizard, an 80-character "I will..." title rule, subcategories and search tags, gig gallery with up to 4 images, FAQ accordions, computed seller cards, post-completion reviews, favorites, order-scoped async messaging with off-platform warning banners, notification preferences, 7 Manage Orders tabs with a computed Late flag, seller onboarding checklists with verification gates, and a comprehensive Wallet view). 2. **Fixes Architectural & Security Defects Found in v1.0:** - **Ledger Invariant Broken:** v1.0 credited escrow without debiting the buyer, resulting in non-zero balance drift. Fixed via a 5-kind `ledger_accounts` table and PostgreSQL deferred zero-sum triggers. - **Auto-Complete Escrow Leak:** v1.0 background completion omitted the escrow debit. Fixed with unified 3-entry settlement transactions. - **File URL Information Leak:** v1.0 returned raw deliverable URLs to buyers during the review phase. Fixed by hiding raw S3 object URLs and exposing only watermarked previews or file metadata/tree listings until order completion. - **Upload Insecurity:** Replaced client-supplied upload URLs with server-signed S3 policies enforcing key path prefixing (`deliveries/{orderId}/{uuid}`), size constraints (≤ 50 MB), and server-side magic-byte verification. - **Session & Invalidation Gaps:** Replaced stateless JWT invalidation claims with a server-backed `sessions` table (`sid` in JWT), rotating refresh cookies, and strict rate-limiting tiers. - **State Machine Concurrency:** Replaced unchecked status updates with optimistic concurrency locking (`UPDATE ... WHERE id = ? AND status = ? RETURNING *`) returning `409 Conflict` on race conditions. 3. **Re-Scopes the Sprint Plan & Balances Team Load:** Eliminates Member 2 idle time in Sprint 1 by front-loading gallery uploads and FAQ UI, and establishes an explicit feature drop order to protect non-negotiables (ledger correctness, IDOR authorization tests, and the background reconciler).

Architectural Continuity: Baseline infrastructure elements (Prisma ORM, Docker Compose, PostgreSQL schema extensions, BullMQ Redis worker configurations, and environment variables) carry over and are expanded in [Appendices A-E](#).

Table of Contents

1. [Vision & Scope](#)
2. [Fiverr Benchmark: What We Adopt, Defer, and Reject](#)
3. [Target Users & Context-Driven Personas](#)
4. [Feature Requirements \(MVP, Revised\) - 4.1 Accounts, Security & Seller Onboarding - 4.2 Gig Catalog & Discovery - 4.3 Order Lifecycle & Workspace - 4.4 Delivery & Asset Security - 4.5 Messaging, Notifications & Reviews - 4.6 Category & Subcategory Taxonomy](#)
5. [Feature Requirements \(Post-MVP Roadmap: V2 & V3\)](#)
6. [Core User Flows & System Sequence](#)
7. [System Architecture: Modular Monolith](#)
8. [Technology Stack & Architectural Decisions](#)
9. [Database Schema & Data Model Changes](#)
10. [Order State Machine \(Revised\)](#)
11. [Double-Entry Escrow Ledger & Wallet Design](#)
12. [API Specification & Contract](#)
13. [Authentication, Authorization & Security Architecture](#)
14. [File Storage & Asset Protection Pipeline](#)
15. [Background Job Processing & Worker System](#)
16. [UI/UX Specifications & Wireframes](#)
17. [Non-Functional Requirements & Operational SLAs](#)
18. [Team Structure & Re-Scoped Sprint Plan](#)
19. [Risk Assessment & Open Decisions](#)
20. [Glossary](#)
21. [Appendices](#)
 - [Appendix A: Environment Variables](#)
 - [Appendix B: Complete Prisma Schema \(Reference\)](#)
 - [Appendix C: Docker Compose \(Local Development\)](#)
 - [Appendix D: Testing & Verification Strategy](#)
 - [Appendix E: Production Deployment Checklist](#)

1. Vision & Scope

1.1 The One-Sentence Pitch

A high-velocity micro-task marketplace where clients purchase fixed-price, bite-sized digital deliverables (strictly under \$50, delivered in 24–48 hours) protected by automated double-entry escrow — with zero bidding, no proposals, and no price negotiation.

1.2 Vision Statement

Micro-Gig Marketplace exists to eradicate the frictional overhead of traditional freelance platforms. Modern freelance networks subject buyers and sellers to lengthy proposal-writing cycles, ambiguous scoping, variable hourly rates, and prolonged settlement delays.

By enforcing the **Service-as-a-Product (SaaP)** paradigm, our platform transforms creative, technical, and operational services into clearly demarcated, off-the-shelf SKU items. Buyers browse, enter structured requirements, escrow payment with a single click, and receive verified deliverables within 24 to 48 hours.

1.3 Key Strategic Differentiator vs. Fiverr

Dimension	Modern Fiverr Platform	Micro-Gig Marketplace (Our Thesis)
Price Spectrum	Wide range (\$10 to \$650+)	Strictly under \$50 (Micro-Gigs only)
Pricing Tiers	3 complex tiers (Basic, Standard, Premium)	Single fixed-price per gig
Delivery Turnaround	Up to 14–30 days	Strictly 24–48 hours
Scoping Process	Custom offers, consultations, buyer briefs	Zero negotiations; standardized requirements form only
Seller Pages	Multi-media, heavy video portfolios, agency bios	Lightweight gallery (max 4 images), structured skill chips
Platform Fee	Dual-sided (20% seller + buyer service fee)	Transparent 20% flat seller-side fee for MVP

1.4 Architecture & System Boundaries

The system is deployed as a **Modular Monolith** using Next.js 14 App Router, TypeScript, Prisma ORM, PostgreSQL, Redis/BullMQ, and S3-compatible cloud storage.

Internally, strict domain module boundaries decouple core concerns: 1. **Catalog Module**: Gig discovery, categories/subcategories, tag filtering, full-text search, and favorites. 2. **Seller Profile Module**: Onboarding checklist, verification stub, skill/language profiles, public seller card. 3. **Order Engine**: State machine lifecycle, deadline enforcement, revision quotas, and dispute routing. 4. **Ledger & Wallet Module**: Double-entry financial accounts, zero-sum trigger validation, fee withholding, and wallet analytics. 5. **Delivery & Asset Module**: S3 pre-signed upload security, metadata extraction, watermarking, and ZIP inspection. 6. **Messaging Module**: Order-scoped async message threads, file attachments, and anti-leakage pattern warnings. 7. **Notification Module**: Bell dropdown, unread count badge, email delivery, and user preference matrix. 8. **Identity & Access Module**: Server-side sessions, JWT cookies with rotation, and contextual permission derivations.

1.5 Project Constraints & Operating Parameters

- **Development Timeline**: 6 weeks total.
- **Engineering Headcount**: 4 full-stack software engineers.
- **Infrastructure Budget**: Free-tier / low-cost managed cloud services (Supabase/Neon PostgreSQL, Upstash Redis, AWS S3/Cloudflare R2, Vercel).
- **Payment Processing**: Simulated payments in USD virtual balances (test wallet top-up and withdrawal) to preserve regulatory exemption during academic and prototyping phases while maintaining 100% production-ready ledger mechanics.

2. Fiverr Benchmark: What We Adopt, Defer, and Reject

A thorough comparative analysis of 48 production Fiverr screenshots established the following engineering and product decisions:

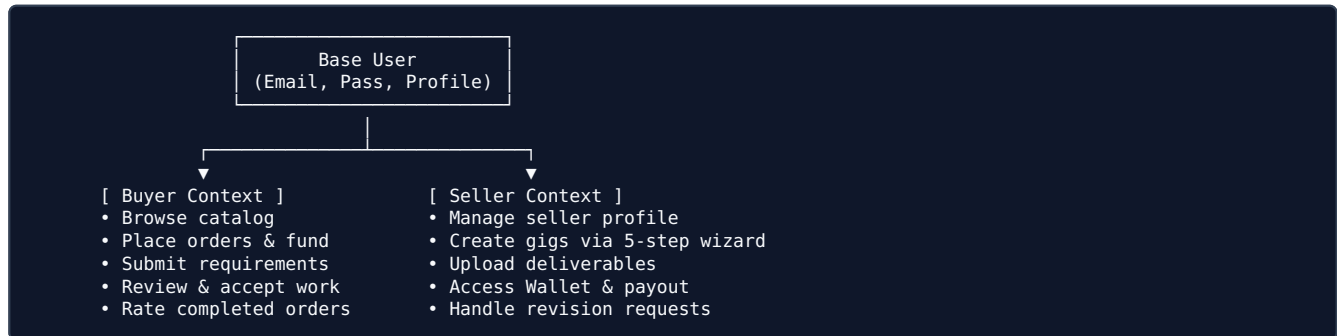
#	Fiverr Feature Observed	Product Decision	Implementation Target
1	Gig Creation Wizard (Overview → Pricing → Description & FAQ → Requirements → Gallery → Publish)	Adopt: 5-step wizard with client autosave. Single fixed price.	MVP
2	Gig Title Rules ("I will..." prefix, 80-char max, subcategory, up to 5 tags)	Adopt: Title capped at 80 chars; enforced "I will..." prefix in UI. Category + subcategory selection + 5 tags.	MVP
3	3-Tier Pricing (Basic / Standard / Premium), add-ons, extra-fast delivery	Defer: Keep single price. Store <code>revisions_included</code> (default 2) now so V2 tiers require zero schema migrations.	MVP (Field) / V2 (Tiers)
4	Gig Gallery & Portfolio	Adopt Gallery: 1 thumbnail + up to 3 showcase images. Defer deep external portfolio integration.	MVP
5	FAQ Accordion on Gig Page	Adopt: Collapsible accordion supporting up to 5 FAQ pairs per gig (<code>gig_faqs</code>).	MVP
6	Seller Card & Badges (Member since, avg response time, last delivery, languages, country)	Adopt: Computed from database events and profile metadata.	MVP
7	Review System (5-star ratings, sub-scores for communication/quality/value, seller reply)	Adopt MVP-Lite: Single 1-5 star rating + text review after order completion. Sub-scores and seller replies deferred.	MVP-LITE
8	Favorites & Lists (Heart count and custom collections)	Adopt Favorites: Toggle heart icon and public favorite count. Custom lists deferred to V2.	MVP
9	Messaging Inbox (Order threads, filtering, attachment sharing, off-platform warnings)	Adopt Order-Scoped: Async message thread per order with 15s polling and anti-leakage banner. Real-time WebSockets deferred.	MVP-LITE
10	Notification Center (In-app bell badge, category email preferences)	Adopt: Bell dropdown with unread badge + email notifications. Preference matrix per category. Push notifications deferred.	MVP
11	Manage Orders Tabs (Priority, Active, Incomplete, Late, Delivered, Completed, Cancelled, Starred)	Adopt: 7 primary tabs with real-time counts. Introduce computed <code>is_late</code> flag.	MVP
12	Seller Levels (Level 0, 1, 2, Top Rated driven by scores, response rates, earnings)	Defer: Capture underlying analytics metrics now; implement automated level promotion logic in V2.	V2 ROADMAP
13	Seller Dashboard & Onboarding Checklist ("Complete these steps to become visible")	Adopt: 4-step checklist. Gigs remain hidden from search until profile and ID stub are complete.	MVP
14	Seller Profile Fields (About 150+ chars, skills, proficiency, languages)	Adopt: About bio (150-600 chars), skills chips, and spoken languages with proficiency levels.	MVP
15	Trust & Safety Verification (ID verification stub, W-9 tax forms)	Adopt Stub: ID verification stub with manual admin approval gate. Skip tax compliance forms.	MVP-STUB
16	Earnings Dashboard (Available funds, pending clearance, in active orders, lifetime earnings, CSV export)	Adopt: Comprehensive Wallet view powered by the double-entry escrow ledger.	MVP
17	Billing History & Payment Receipts	Adopt MVP-Lite: Ledger-backed transaction history with printable simulated invoices.	MVP-LITE
18	Seller Analytics (Earnings, average selling price, on-time delivery rate, completion rate)	Adopt Headline Stats: Four core metrics calculated on-the-fly. Advanced historical charts deferred to V3.	MVP-LITE
19	Account Security (Password change, active sessions list, remote logout-all)	Adopt: Server-side session tracking with immediate remote session revocation. TOTP 2FA deferred to V2.	MVP / V2
20	Recommendation Carousels ("More from this seller", "Recommended for you")	Adopt Single Strip: "More from this seller" (up to 5 gigs) on gig detail page. ML recommendation engines deferred.	MVP
21	Custom Offers & Hourly Contracts (Fiverr Pro, Mira AI briefs, buyer requests)	Reject: Explicitly conflicts with our core product thesis of fixed-price, instant-checkout micro-tasks.	OUT OF SCOPE
22	Trust Badges	Adopt Single Badge: "Verified" badge awarded upon admin approval of seller identity stub.	MVP

3. Target Users & Context-Driven Personas

3.1 Single User Entity with Contextual Roles

A fundamental flaw of legacy architectures is partitioning user tables into rigid `buyers` and `sellers`. Following Fiverr's operational model, our system establishes a unified `users` table.

Every authenticated account has the intrinsic capability to buy services, manage purchases, and receive deliverables. A user gains **Seller Capabilities** dynamically when they complete their seller onboarding profile and receive identity approval. System administrators are designated via a strict `is_admin` boolean flag with server-enforced audit logging.



3.2 Detailed User Personas

Persona 1: Alice — The Time-Constrained Founder (Buyer)

- **Background:** Solo founder building a SaaS product. She lacks advanced graphic design and front-end CSS debugging skills.
- **Pain Points:** Upwork and Fiverr require sorting through dozens of vague proposals, navigating complex 3-tiered packages, or waiting 5–7 days for simple fixes.
- **Jobs to be Done:** Find a vetted freelancer to fix a broken mobile navigation CSS bug or generate a sharp app icon within 24 hours for a flat fee under \$30.
- **Success Criteria:** Instant checkout with clear requirement prompts, watermarked delivery preview, and one-click completion.

Persona 2: David — The Rapid-Turnaround Specialist (Seller)

- **Background:** Junior web developer and digital designer seeking predictable side income.
- **Pain Points:** Writing custom pitches and bidding against low-cost agencies wastes hours of unbillable time. Clients frequently attempt scope creep.
- **Jobs to be Done:** Package specific modular skills ("I will fix 1 responsive CSS bug", "I will design 1 modern vector app icon") for a fixed \$20–\$40 payout with strict 24-hour delivery expectations.
- **Success Criteria:** Standardized requirement briefs upfront, clear 2-revision limit, and guaranteed automated escrow payout after 72 hours of buyer delivery inactivity.

Persona 3: Marcus — The Trust & Safety Administrator (Admin)

- **Background:** Internal operations specialist responsible for marketplace integrity.
- **Pain Points:** Fraudulent accounts, off-platform payment poaching, chargebacks, and deadlocked client disputes.
- **Jobs to be Done:** Review submitted seller ID verifications, arbitrate flagged order disputes (authorizing full buyer refund, full seller release, or partial split), and verify ledger zero-sum balance integrity.
- **Success Criteria:** Dedicated admin audit dashboard, non-repudiable event logs, and single-click dispute resolution.

4. Feature Requirements (MVP, Revised)

The feature matrix is categorized by functional domain and prioritized: - **P0 (Must Ship)**: Core operational path required for transactional integrity. - **P1 (Should Ship)**: High-value usability, safety, and engagement features. - **NEW / CHANGED**: Specific modifications introduced in v1.1 resulting from the Fiverr benchmark and defect remediation.

4.1 Accounts, Security & Seller Onboarding

Feature ID	Status	Priority	Description	Acceptance Criteria
AUTH-01	UNCHANGED	P0 MUST SHIP	Email & Password Registration	Users register with email, password (min 8 chars, 1 number, 1 symbol), and full name. Verifies unique email address.
AUTH-02	UNCHANGED	P0 MUST SHIP	Credential Authentication	Login validates against <code>bcrypt</code> hash (cost factor 12). Issues a secure, signed JWT.
AUTH-03	UNCHANGED	P0 MUST SHIP	HTTP-Only Cookie Storage	Auth tokens are stored exclusively in <code>HttpOnly</code> , <code>SameSite=Lax</code> , <code>Secure</code> cookies. Zero local storage token exposure.
AUTH-04	UNCHANGED	P0 MUST SHIP	User Logout	Clears the auth cookie and invalidates the session record.
AUTH-05	CHANGED	P0 MUST SHIP	Derived Capability Authorization	Eliminates the static <code>users.role</code> enum from v1.0. User authorization is evaluated contextually: <code>is_admin</code> handles platform administration; seller permissions are derived dynamically if the user has an approved, active <code>seller_profiles</code> record.
AUTH-06	UNCHANGED	P0 MUST SHIP	Password Hashing Standard	Enforces <code>bcrypt</code> with salt rounds = 12 on all credential writes.
AUTH-07	UNCHANGED	P0 MUST SHIP	Client-Side Auth State Synchronization	Provides a lightweight <code>/api/v1/auth/me</code> endpoint returning <code>{ id, email, fullName, isAdmin, isSeller }</code> for UI state hydration.
AUTH-08	NEW IN V1.1	P1 SHOULD SHIP	Session Management & Remote Logout	Users can view active login sessions (<code>sessions</code> table tracking user agent, IP address, and creation date) and trigger "Log Out Everywhere" to instantly revoke all active tokens.
SEL-01	NEW IN V1.1	P0 MUST SHIP	Structured Seller Profile	Allows a user to create a seller identity: Display Name, Headline (max 80 chars), About bio (min 150, max 600 chars), Country, Spoken Languages (with proficiency: Basic, Conversational, Fluent, Native), and Technical Skills (with experience level: Beginner, Intermediate, Expert).
SEL-02	NEW IN V1.1	P0 MUST SHIP	Seller Onboarding Checklist Gate	Seller dashboard displays a sequential 4-step onboarding checklist: (1) Complete Profile → (2) Submit ID Verification → (3) Create First Gig → (4) Publish. Seller gigs remain strictly hidden from catalog search until steps 1 and 2 are approved.
SEL-03	NEW IN V1.1	P1 SHOULD SHIP	Public Seller Card	Dynamically calculates and renders public credibility stats on gig and seller pages: Member since date, Average Response Time (hours), Last Delivery Date, and Lifetime Order Completion Rate (%).

4.2 Gig Catalog & Discovery

Feature ID	Status	Priority	Description	Acceptance Criteria
GIG-01	CHANGED	P0 MUST SHIP	Fiverr-Aligned Gig Schema & Wizard	Gig creation enforces: (1) Title max 80 chars starting with mandatory "I will..." prefix in UI, (2) Mandatory Category & Subcategory selection, (3) Up to 5 search tags, (4) Explicit <code>revisions_included</code> count (integer 0-5, default 2), (5) Gallery of 1 primary thumbnail + up to 3 showcase images, and (6) Up to 5 collapsible FAQ pairs (<code>gig_faqs</code>).
GIG-02	UNCHANGED	P0 MUST SHIP	Single Fixed-Price Model	Price must be an integer between \$5.00 and \$50.00. No multi-tier pricing in MVP.
GIG-03	UNCHANGED	P0 MUST SHIP	Turnaround Time Limit	Delivery timeframe must be strictly 24 hours or 48 hours.
GIG-04	UNCHANGED	P0 MUST SHIP	Structured Buyer Requirements Prompt	Seller defines 1 to 3 specific prompt questions that the buyer must answer upon ordering.
GIG-05	UNCHANGED	P0 MUST SHIP	Public Marketplace Catalog View	Renders a responsive grid of published gigs displaying thumbnail, title, seller avatar, turnaround badge, rating, and price.
GIG-06	CHANGED	P0 MUST SHIP	Multi-Facet Search & Filter	Search supports keyword matching, category filtering, subcategory filtering, and sorting by: <code>price_asc</code> , <code>price_desc</code> , <code>rating_desc</code> , and <code>delivery_time_asc</code> .
GIG-07	UNCHANGED	P0 MUST SHIP	Public Gig Detail Page	Comprehensive view rendering title, media gallery carousel, seller card, full markdown description, FAQ accordion, reviews, and sticky order CTA.
GIG-08	UNCHANGED	P0 MUST SHIP	Seller Gig State Management	Seller can toggle gig status between <code>PUBLISHED</code> and <code>PAUSED</code> .
GIG-09	UNCHANGED	P0 MUST SHIP	Auto-Generated Slug Routing	Gigs generate SEO-friendly slugs derived from title and nanoid (e.g., <code>/gigs/i-will-fix-your-css-layout-bug-a8f9</code>).
GIG-10	NEW IN V1.1	P1 SHOULD SHIP	Gig Favoriting & Social Proof	Authenticated buyers can favorite/unfavorite gigs with heart icons. Total favorite count is displayed on the gig card and detail page.
GIG-11	NEW IN V1.1	P1 SHOULD SHIP	"More From This Seller" Strip	Gig detail page displays an auxiliary horizontal carousel showing up to 5 additional active gigs from the same seller.
GIG-12	NEW IN V1.1	P0 MUST SHIP	Category & Subcategory Hierarchy	Dedicated category landing pages displaying an interactive subcategory tree, featured micro-gigs, and active filter tags.

4.3 Order Lifecycle & Workspace

Feature ID	Status	Priority	Description	Acceptance Criteria
ORD-01	UNCHANGED	P0 MUST SHIP	One-Click Order Placement	Buyer initiates order from gig page. System debits buyer virtual wallet, credits platform escrow, and generates an order in <code>PENDING_REQUIREMENTS</code> status.
ORD-02	UNCHANGED	P0 MUST SHIP	Requirements Submission Gate	Buyer fills required prompt fields. Order transitions to <code>IN_PROGRESS</code> , and the delivery countdown timer activates.
ORD-03	UNCHANGED	P0 MUST SHIP	Order Workspace Dashboard	Dedicated workspace <code>/orders/[id]</code> featuring order status stepper, live countdown timer, requirements brief, delivery history, async messaging tab, and activity log.
ORD-04	UNCHANGED	P0 MUST SHIP	Seller Deliverable Submission	Seller uploads deliverable file key and notes. Order transitions from <code>IN_PROGRESS</code> or <code>IN_REVISION</code> to <code>DELIVERED</code> .
ORD-05	UNCHANGED	P0 MUST SHIP	Buyer Delivery Review	Buyer inspects delivery notes, checksum, file-tree, or watermarked preview image.
ORD-06	UNCHANGED	P0 MUST SHIP	Buyer Acceptance & Escrow Release	Buyer clicks "Accept Delivery". Order transitions to <code>COMPLETED</code> . Ledger debits escrow, credits seller pending balance, and books platform revenue.
ORD-07	UNCHANGED	P0 MUST SHIP	Buyer Revision Request	Buyer submits revision feedback. Order transitions to <code>IN_REVISION</code> .
ORD-08	UNCHANGED	P0 MUST SHIP	Auto-Completion Time Gate	If buyer takes zero action for 72 hours following delivery, BullMQ background job transitions order to <code>COMPLETED</code> and releases escrow.
ORD-09	UNCHANGED	P0 MUST SHIP	Order Cancellation (Unstarted)	Buyer may cancel an order still in <code>PENDING_REQUIREMENTS</code> . Escrow is refunded 100% to buyer wallet.
ORD-10	UNCHANGED	P0 MUST SHIP	Mutual Cancellation Request	Either party may propose mutual cancellation while <code>IN_PROGRESS</code> . Counterparty approval triggers 100% refund.
ORD-11	NEW IN V1.1	P0 MUST SHIP	Computed Late Flag	An order is dynamically evaluated as <code>is_late = true</code> when <code>current_timestamp > deadline</code> AND status is <code>IN_PROGRESS</code> or <code>IN_REVISION</code> . Displayed prominently via red warning badges.
ORD-12	NEW IN V1.1	P0 MUST SHIP	Manage Orders 7-Tab Navigation	Dedicated order management dashboard <code>/orders</code> featuring 7 filtered tabs with badge counts: <code>Priority</code> (Late + Delivered awaiting buyer action), <code>Active</code> , <code>Late</code> , <code>Delivered</code> , <code>Completed</code> , <code>Cancelled</code> , and <code>Starred</code> .
ORD-13	NEW IN V1.1	P0 MUST SHIP	Strict Revision Quota Enforcement	Buyer cannot submit more revisions than <code>revisions_included</code> (default 2). Once exhausted, the revision CTA is disabled, leaving only "Accept Delivery" or "Open Dispute".
ORD-14	NEW IN V1.1	P1 SHOULD SHIP	Formal Order Dispute Workflow	Either party can flag an order as <code>DISPUTED</code> from <code>DELIVERED</code> or <code>IN_PROGRESS</code> . Order enters admin triage queue; administrator resolves with full buyer refund, full seller release, or custom split.
ORD-15	NEW IN V1.1	P1 SHOULD SHIP	24-Hour Late Delivery Buyer Remedy	If an order remains undelivered 24 hours past the agreed deadline, the buyer receives an automated unilateral "Cancel & Refund" action button.
ORD-16	NEW IN V1.1	P1 SHOULD SHIP	Order Star / Bookmarking	Users can star individual orders for priority tracking, stored in <code>order_stars</code> .

4.4 Delivery & Asset Security

Feature ID	Status	Priority	Description	Acceptance Criteria
DEL-01	UNCHANGED	P0 MUST SHIP	S3 Pre-Signed Upload Flow	Backend generates signed S3 PUT policy. Client uploads directly to cloud storage, bypassing web app server memory.
DEL-02	CHANGED	P0 MUST SHIP	Secure Server-Generated Key Path	Server strictly determines the target S3 key pattern: <code>deliveries/{orderId}/{deliveryUuid}/{filename}</code> . Client-supplied target keys are rejected.
DEL-03	CHANGED	P0 MUST SHIP	Pre-Signed Upload Policy Restrictions	Policy enforces strict <code>Content-Type</code> matching and max file size (50 MB).
DEL-04	UNCHANGED	P0 MUST SHIP	Direct Cloudflare R2 / AWS S3 Storage	Deliverables stored in private, unguessable object paths without public read permissions.
DEL-05	UNCHANGED	P0 MUST SHIP	Expiring Download Links	Deliverable download links use pre-signed GET URLs with a 15-minute TTL.
DEL-06	UNCHANGED	P1 SHOULD SHIP	Image Watermark Generation	Images (PNG, JPG, WEBP) trigger a background Sharp pipeline that superimposes diagonal translucent watermark text across the preview image.
DEL-07	NEW IN V1.1	P0 MUST SHIP	Non-Image Deliverable Protection	For non-image deliverables (ZIP, code snippets, PDF, documents) where watermarking is impossible, raw download is locked until <code>COMPLETED</code> . Buyer is provided filename, byte size, SHA-256 integrity hash, and a sanitized directory tree listing for archive files.

4.5 Messaging, Notifications & Reviews

Feature ID	Status	Priority	Description	Acceptance Criteria
MSG-01	NEW IN V1.1	P0 MUST SHIP	Order-Scoped Async Message Thread	Every order maintains a dedicated message thread (<code>messages</code> table) accessible only to the buyer, seller, and admin. Supports text and attachments up to 10 MB. Polling every 15 seconds (no WebSockets in MVP).
MSG-02	NEW IN V1.1	P1 SHOULD SHIP	Messaging Inbox & Filters	Inbox interface displaying all active conversation threads with tab filters: <code>All</code> , <code>Unread</code> , <code>Starred</code> , and <code>Archived</code> .
MSG-03	NEW IN V1.1	P1 SHOULD SHIP	Anti-Disintermediation Warning Banner	Displays a persistent UI warning banner: "Keep payments and communication on the platform." Client-side and server-side regex scans for external contact attempts (email, phone, Telegram, WhatsApp, PayPal) and renders a warning prompt before sending (warn only; does not block).
NOT-01	NEW IN V1.1	P0 MUST SHIP	Granular Event Notification Triggers	System dispatches notifications on 9 key lifecycle events: (1) Order Placed, (2) Requirements Submitted, (3) Deliverable Uploaded, (4) Revision Requested, (5) Order Completed, (6) Auto-Completion 24h Warning, (7) Late Delivery Warning (50% and 90% SLA elapsed), (8) Dispute Opened, and (9) New Order Message.
NOT-02	NEW IN V1.1	P0 MUST SHIP	Dual-Channel Delivery & Preferences	Delivers via in-app notification bell (with unread count badge) and transactional email. User preference matrix allows toggling email alerts per category (in-app alerts are permanently on for critical order events).
REV-01	NEW IN V1.1	P1 SHOULD SHIP	Post-Completion Buyer Rating & Review	Upon reaching <code>COMPLETED</code> , buyer has a 14-day window to leave one 1-5 star rating and an optional text review (max 500 chars). Stored in <code>reviews</code> table and factored into gig and seller aggregate scores.
REV-02	NEW IN V1.1	P1 SHOULD SHIP	Automated Rating Reminder	Dispatches a gentle notification and email reminder to the buyer 24 hours after order completion if no review has been submitted.

4.6 Category & Subcategory Taxonomy

Informed by Fiverr's marketplace taxonomy, our catalog organizes micro-services into 6 foundational categories and standardized subcategories:

```

├── Graphics & Design
│   ├── Logo Design & Tweaks
│   ├── Flyer & Social Media Banners
│   ├── Business Card Layouts
│   ├── Vector App Icon Design
│   └── Background Removal & Photo Retouching
├── Programming & Tech
│   ├── Bug Fixes (HTML/CSS & JavaScript)
│   ├── Responsive Mobile Layout Fix
│   ├── Single Landing Page Implementation
│   ├── Script & Automation Quick Fix
│   └── API Webhook Integration
├── Writing & Translation
│   ├── Proofreading (Up to 1,000 words)
│   ├── Technical Documentation Polish
│   ├── Product Description Copy
│   ├── Resume & Cover Letter Tweak
│   └── English / Spanish / French Short Translation
├── Video & Animation
│   ├── Short Video Trim & Cut (Under 60s)
│   ├── Subtitle & Caption Burn-in
│   ├── Intro / Outro Logo Animation
│   └── Video Audio Noise Cleanup
├── Digital Marketing
│   ├── Website Technical SEO Audit
│   ├── Social Media Copywriting (5 Posts)
│   ├── Email Newsletter Template Polish
│   ├── Meta Tag & OpenGraph Optimization
├── Operations & Admin
│   ├── Data Entry & CSV Cleaning (Up to 500 rows)
│   ├── PDF to Word / Markdown Conversion
│   └── Virtual Assistant Task (1 Hour Fixed Scope)

```

5. Feature Requirements (Post-MVP Roadmap: V2 & V3)

The following capabilities are deliberately staged for subsequent iterations to preserve MVP delivery velocity while maintaining zero-friction migration paths:

Phase 2: Trust, Scalability & Depth (Weeks 7-12)

- **Multi-Tier Pricing (Basic, Standard, Premium):** Expand gig pricing from a single rate to three discrete tiers with varying deliverables and turnaround times (`revisions_included` field is already provisioned in v1.1 database schema).
- **Gig Extra Add-ons:** Allow sellers to attach optional add-ons (e.g., "Fast 12-hour turnaround: +\$15", "Source file inclusion: +\$10").
- **Seller Levels & Automated Promotion:** Automatically promote sellers from Level 0 to Level 1, 2, and Top Rated based on trailing 60-day metrics (orders completed, average rating ≥ 4.7 , response rate $\geq 90\%$, earnings).
- **Real-Time WebSockets Messaging:** Migrate order message threads and notifications from 15-second HTTP polling to bi-directional WebSockets via Pusher or Socket.io.
- **Two-Factor Authentication (TOTP):** Support authenticator apps (Google Authenticator, Authy) with backup recovery codes.
- **Custom Buyer Lists:** Allow buyers to organize favorited gigs into named private and shareable collections.
- **Public Seller Portfolio:** Enable sellers to attach verified project deliverables directly to a showcase portfolio tab on their profile.
- **Review Sub-Scores & Public Seller Replies:** Expand review feedback to 3 sub-metrics (Service as Described, Communication, Value for Money) and provide sellers a 1-time public response right.

Phase 3: Marketplace Expansion & Automation (Weeks 13-20)

- **Real Stripe Connect & PayPal Escrow Integration:** Transition from simulated virtual wallets to live automated fiat disbursement via Stripe Connect Custom / Express onboarding.

- **Full-Text Elastic / MeiliSearch Integration:** Index gigs and seller profiles for typo-tolerant fuzzy matching, automated keyword highlighting, and semantic search.
- **Automated Anti-Fraud & Phishing ML Classifier:** Replace regex pattern matching with real-time NLP classification to detect attempted contact information leakage and external transaction poaching.
- **Automated PDF / Video Watermark Pipeline:** Serverless AWS Lambda / FFmpeg pipelines for dynamic watermarking across multi-page PDF documents and MP4/MOV video files.
- **Advanced Seller Analytics:** Historical revenue charts, keyword impression counters, conversion funnel analysis, and comparative benchmark metrics.

6. Core User Flows & System Sequence

6.1 Primary Flow: Buyer Discovery, Order Placement & Requirements

SEQUENCE DIAGRAM Core System Interaction Flow

```
sequenceDiagram
    autonumber
    actor Buyer
    participant Web as Next.js Client
    participant API as Order Route Handler
    participant DB as PostgreSQL
    participant Ledger as Ledger Engine

    Buyer->>Web: Browse Catalog / Filter by Category & Tags
    Web->>Buyer: Render Gig Card (Thumbnail, Title, Price, Rating, Turnaround)
    Buyer->>Web: Select Gig & Click "Continue to Order"
    Web->>Buyer: Display Checkout Review ($35.00 total)
    Buyer->>Web: Confirm Purchase (Idempotency-Key: uuid)
    Web->>API: POST /api/v1/orders { gigId, idempotencyKey }

    rect rgb(240, 248, 255)
        Note over API,Ledger: Atomic Database Transaction
        API->>DB: Check Buyer USER_AVAILABLE Balance >= $35.00
        alt Balance Insufficient
            API-->>Web: 402 Payment Required (Prompt Virtual Top-Up)
        else Balance Sufficient
            API->>Ledger: Transfer $35.00: BUYER AVAILABLE (-) -> ESCROW (+)
            API->>DB: Insert Order (status: PENDING_REQUIREMENTS)
            API->>DB: Insert order_events (ORDER_PLACED)
        end
    end

    API-->>Web: 201 Created { orderId, status: "PENDING_REQUIREMENTS" }
    Web->>Buyer: Redirect to /orders/[id] (Display Requirements Prompt)
    Buyer->>Web: Enter Bug Description + GitHub Repository URL
    Web->>API: POST /api/v1/orders/[id]/requirements { answers }
    API->>DB: Update Order status: IN_PROGRESS, set deadline = now() + 24h
    API->>DB: Insert order_events (REQUIREMENTS_SUBMITTED)
    API-->>Web: 200 OK (Countdown Timer Commences)
    Web->>Buyer: Render Active Order Workspace
```

6.2 Secondary Flow: Delivery, Revision Quota & Escrow Release

SEQUENCE DIAGRAM Core System Interaction Flow

```
sequenceDiagram
    autonumber
    actor Seller
    actor Buyer
    participant Web as Next.js Client
    participant API as Fulfillment Handler
    participant S3 as S3 Cloud Store
    participant Worker as BullMQ Worker
    participant Ledger as Ledger Engine

    Note over Seller,Web: Order is IN_PROGRESS (Countdown Active)
    Seller->>Web: Open Order Workspace & Click "Deliver Completed Work"
    Web->>API: POST /api/v1/deliveries/presign-upload { filename, contentType, size }
    API->>Web: Return Signed S3 Policy { fileKey: "deliveries/ord_123/uuid/patch.zip", uploadUrl }
    Web->>S3: Direct PUT Binary (Max 50MB)
    Web->>API: POST /api/v1/orders/[id]/deliveries { fileKey, notes }
    API->>API: Verify S3 Metadata & File Integrity
    API->>Worker: Enqueue Auto-Complete Job (Delay: 72 Hours)
    Note over Worker: Job ID: autocomplate:ord_123:del_456
    API->>Web: Order Status: DELIVERED (Deadline Clock Paused)

    alt Scenario A: Buyer Accepts Delivery
        Buyer->>Web: Inspect Delivery Notes, Checksum & File Tree
        Buyer->>Web: Click "Accept & Complete Order"
        Web->>API: POST /api/v1/orders/[id]/complete
        API->>Worker: Cancel Auto-Complete Job
        API->>Ledger: Settle $35.00 Escrow: ESCROW (-$35), SELLER_PENDING (+$28), PLATFORM_REVENUE (+$7)
        API->>Worker: Enqueue 3-Day Clearing Job for Seller Pending Balance
        API->>Web: Status: COMPLETED
        Web->>Buyer: Display 1-5 Star Rating & Review Modal
    else Scenario B: Buyer Requests Revision (Revisions Used < Limit)
        Buyer->>Web: Click "Request Revision" & Enter Detailed Feedback
        Web->>API: POST /api/v1/orders/[id]/revision { feedback }
        API->>Worker: Cancel Auto-Complete Job
        API->>API: Increment revisions used (1/2), Set revision_deadline = now() + 24h
        API->>Web: Status: IN_REVISION (Clock Resumes)
    else Scenario C: Buyer Inactivity (72 Hours Elapsed)
        Worker->>API: Fire autocomplate:ord_123:del_456 Job
        API->>Ledger: Execute Standard Settlement (ESCROW -$35 -> SELLER_PENDING +$28 -> PLATFORM +$7)
        API->>API: Status: COMPLETED (Auto-Resolved)
    end
```

6.3 Secondary Flow: 24-Hour Late Delivery Buyer Remedy

SEQUENCE DIAGRAM Core System Interaction Flow

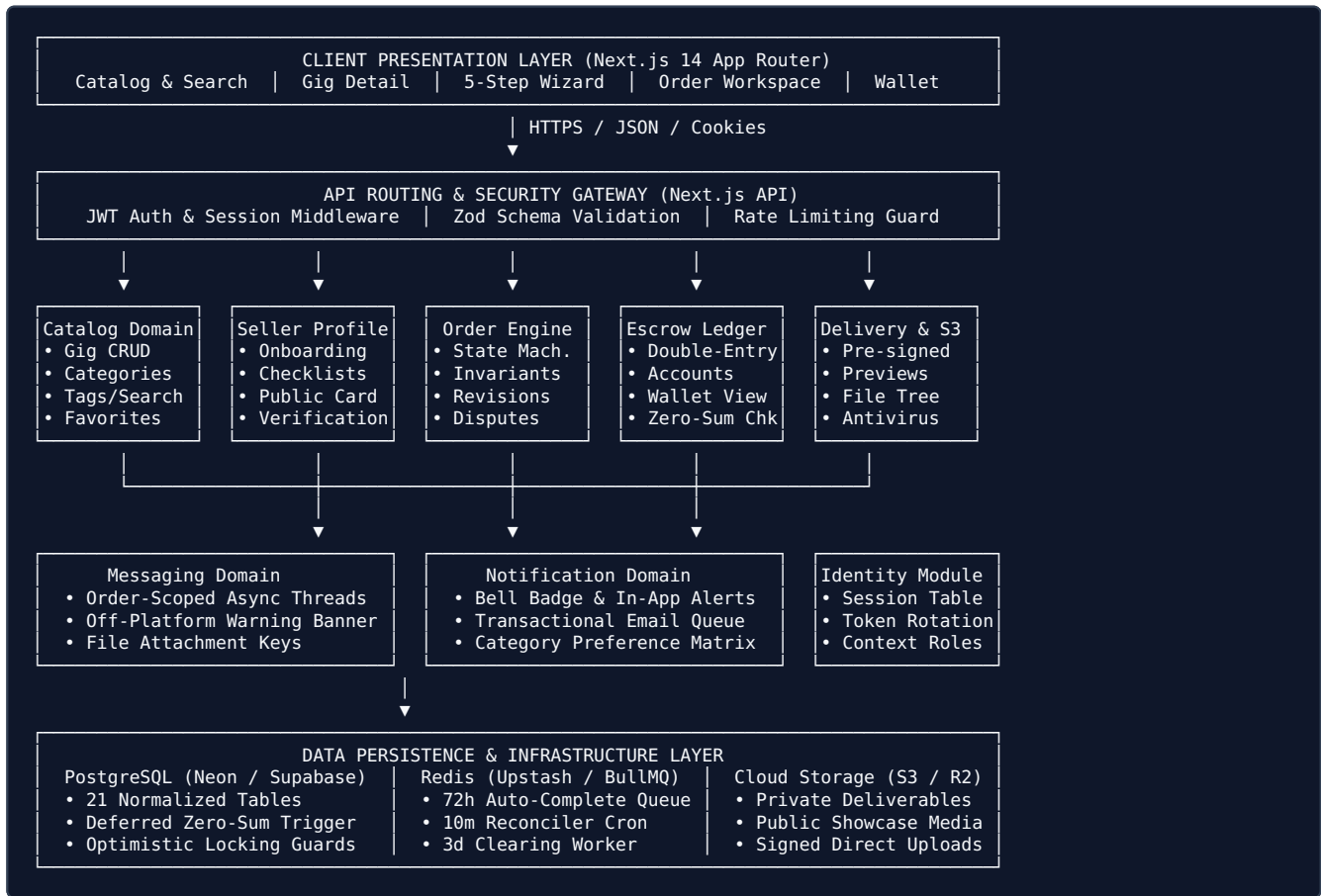
```
sequenceDiagram
    autonumber
    actor Buyer
    participant Web as Next.js Client
    participant API as Order Engine
    participant DB as PostgreSQL
    participant Ledger as Ledger Engine

    Note over Buyer,DB: Status is IN_PROGRESS or IN_REVISION
    Note over Buyer,DB: Current Time > Deadline + 24 Hours
    Buyer->>Web: Open Order Workspace
    Web->>Web: Evaluate is late == true AND now() > deadline + 24h
    Web->>Buyer: Evaluate Red Banner & "Cancel & Refund Order" Action Button
    Buyer->>Web: Click "Cancel Order"
    Web->>API: POST /api/v1/orders/[id]/cancel { reason: "LATE_DELIVERY_24H" }
    API->>DB: Verify now() > deadline + 24h AND status in (IN_PROGRESS, IN_REVISION)
    API->>Ledger: Full Refund: ESCROW (-$35) -> BUYER_AVAILABLE (+$35)
    API->>DB: Update Order status: CANCELLED
    API->>DB: Insert order_events (CANCELLED_LATE_DELIVERY)
    API->>Web: 200 OK { status: "CANCELLED" }
    Web->>Buyer: Display Cancellation Confirmation & Restored Wallet Balance ($35)
```

7. System Architecture: Modular Monolith

7.1 Architectural Blueprint

To achieve production resilience without the operational fragility, network latency, and distributed transaction complexity of microservices, Micro-Gig Marketplace adopts a **Modular Monolith** architecture:



7.2 Domain Module Boundaries & Responsibilities

- Catalog Domain (src/modules/catalog)**: - Manages gig lifecycle: draft, publish, pause, soft-delete. - Enforces the 5-step wizard validation rules and 80-character title schema. - Indexes and filters gigs by category, subcategory, search tags, pricing, and ratings. - Manages buyer favorites and generates the "More from this seller" recommendation strip.
- Seller Profile Domain (src/modules/seller)**: - Manages seller bios, skills with proficiency ratings, and spoken languages. - Drives the 4-step onboarding checklist and locks gig publishing behind ID verification. - Computes public seller card metrics: member age, response time, last delivery, completion rate.
- Order Engine Domain (src/modules/orders)**: - Executes the deterministic state machine transitions with optimistic database locks. - Computes dynamic status flags (`is_late = true`). - Populates the 7 Manage Orders tabs with live counter badges. - Manages buyer requirement submissions, revision limits, and dispute initiation.
- Escrow Ledger & Wallet Domain (src/modules/ledger)**: - Maintains transactional double-entry financial integrity across 5 account kinds. - Enforces PostgreSQL deferred triggers to guarantee zero-sum transactions ($\$sum = 0\$$). - Calculates seller earnings, platform commission (20%), and 3-day clearing periods. - Powers the real-time Wallet view: Available, Pending, In Active Orders, and CSV export.
- Delivery & Asset Domain (src/modules/delivery)**: - Generates secure S3 pre-signed upload policies bound to server-generated key paths. - Enforces file type matching, size quotas (≤ 50 MB), and server-

side magic byte inspection. - Processes image watermarking and generates sanitized directory file-trees for ZIP archives. - Restricts raw asset download URLs from buyers until orders achieve `COMPLETED` status.

- 6. **Messaging Domain (`src/modules/messaging`)**: - Isolates communications into order-scoped conversation threads. - Exposes REST endpoints polled at 15-second intervals. - Analyzes message payloads with non-blocking regex filters to warn on off-platform poaching.
- 7. **Notification Domain (`src/modules/notifications`)**: - Generates persistent in-app alerts and updates the header bell unread counter badge. - Dispatches transactional emails through background BullMQ workers. - Respects user notification preference settings per notification category.
- 8. **Identity & Access Domain (`src/modules/auth`)**: - Issues and verifies signed JWTs stored in `HttpOnly`, `SameSite=Lax` cookies. - Maintains a server-backed `sessions` table supporting remote token invalidation ("Log Out Everywhere"). - Enforces object-level authorization (IDOR checks) verifying user relationship to requested resources.

8. Technology Stack & Architectural Decisions

8.1 Technology Matrix

Layer	Technology	Version	Justification / Strategic Decision
Frontend Framework	Next.js (App Router)	14.2.x	Hybrid SSR/SSG for lightning-fast catalog discovery and SEO; React Server Components minimize bundle weight.
Language	TypeScript	5.4.x	End-to-end static type safety shared across client components, API routes, and database models.
Styling & Icons	Tailwind CSS & Lucide Icons	3.4.x	Rapid design system implementation matching Fiverr's clean aesthetic; Lucide provides clear, lightweight iconography.
Component Library	Shadcn UI (Radix Primitives)	Latest	Accessible, unstyled UI primitives customized for our design system; zero runtime overhead.
Database	PostgreSQL	15+	Rock-solid relational integrity, JSONB support for order events, and deferred constraint triggers for ledger safety.
ORM & Migrations	Prisma ORM	5.14.x	Type-safe query building, declarative schema modeling, and robust automated SQL migration generation.
Cache & Queues	Redis & BullMQ	Redis 7 / BullMQ 5	High-throughput in-memory queue management for delayed auto-completion jobs, notifications, and 10-minute reconcilers.
Object Storage	AWS S3 / Cloudflare R2	S3 API	Secure, scalable storage for gig gallery images and private order deliverables with pre-signed URL access.
Image Processing	Sharp	0.33.x	High-performance native image manipulation for thumbnail generation and diagonal preview watermarking.
Schema Validation	Zod	3.23.x	Strict runtime type inference and request body validation across all client forms and backend route handlers.

8.2 Strategic Architectural Decisions (ADRs)

ADR-01: Modular Monolith vs. Distributed Microservices

- **Context:** A 4-person engineering team with a 6-week delivery constraint cannot afford the operational overhead of microservices (distributed tracing, network partitions, multi-repo synchronization, Kubernetes).
- **Decision:** Build a single Next.js application with strictly enforced internal domain module boundaries (`src/modules/*`).
- **Consequence:** Ultra-fast development speed, unified transactions within PostgreSQL, single-command deployment, zero microservice network latency.

ADR-02: Simulated Payments with Production-Grade Ledger Mechanics

- **Context:** Integrating live payment processors (Stripe Connect) introduces regulatory hurdles, KYC delays, and financial liability during prototype testing.
- **Decision:** Model payment processing via simulated USD virtual balances, backed by a strict, double-entry financial ledger with deferred zero-sum triggers.
- **Consequence:** The application logic and financial mechanics are 100% production-ready. Switching to Stripe in Phase 2 requires only swapping the top-up and withdrawal endpoints.

ADR-03: BullMQ Delayed Jobs + 10-Minute Polling Reconciler

- **Context:** The platform promises automated 72-hour delivery auto-completion. In v1.0, this relied entirely on BullMQ delayed jobs. However, if Redis restarts, jobs can be dropped.
- **Decision:** Use deterministic BullMQ job IDs (`autocomplete:{orderId}:{deliveryId}`) for primary scheduling, backed by a lightweight cron reconciler running every 10 minutes that queries PostgreSQL for any `DELIVERED` order where `auto_complete_at < NOW()`.
- **Consequence:** 100% SLA guarantee on order resolution with zero stranded escrow balances, even in the event of total Redis failure.

9. Database Schema & Data Model Changes

9.1 Summary of v1.1 Schema Evolutions

Version 1.1 refines the persistence layer to support the Fiverr benchmark features, resolve v1.0 design flaws, and guarantee financial and audit integrity:

16 NEW TABLES INTRODUCED		
1. seller_profiles	7. favorites	12. notification_prefs
2. seller_skills	8. order_events	13. reviews
3. user_languages	9. order_stars	14. sessions
4. gig_faqs	10. messages	15. ledger_accounts
5. gig_images	11. notifications	16. disputes
6. gig_tags		

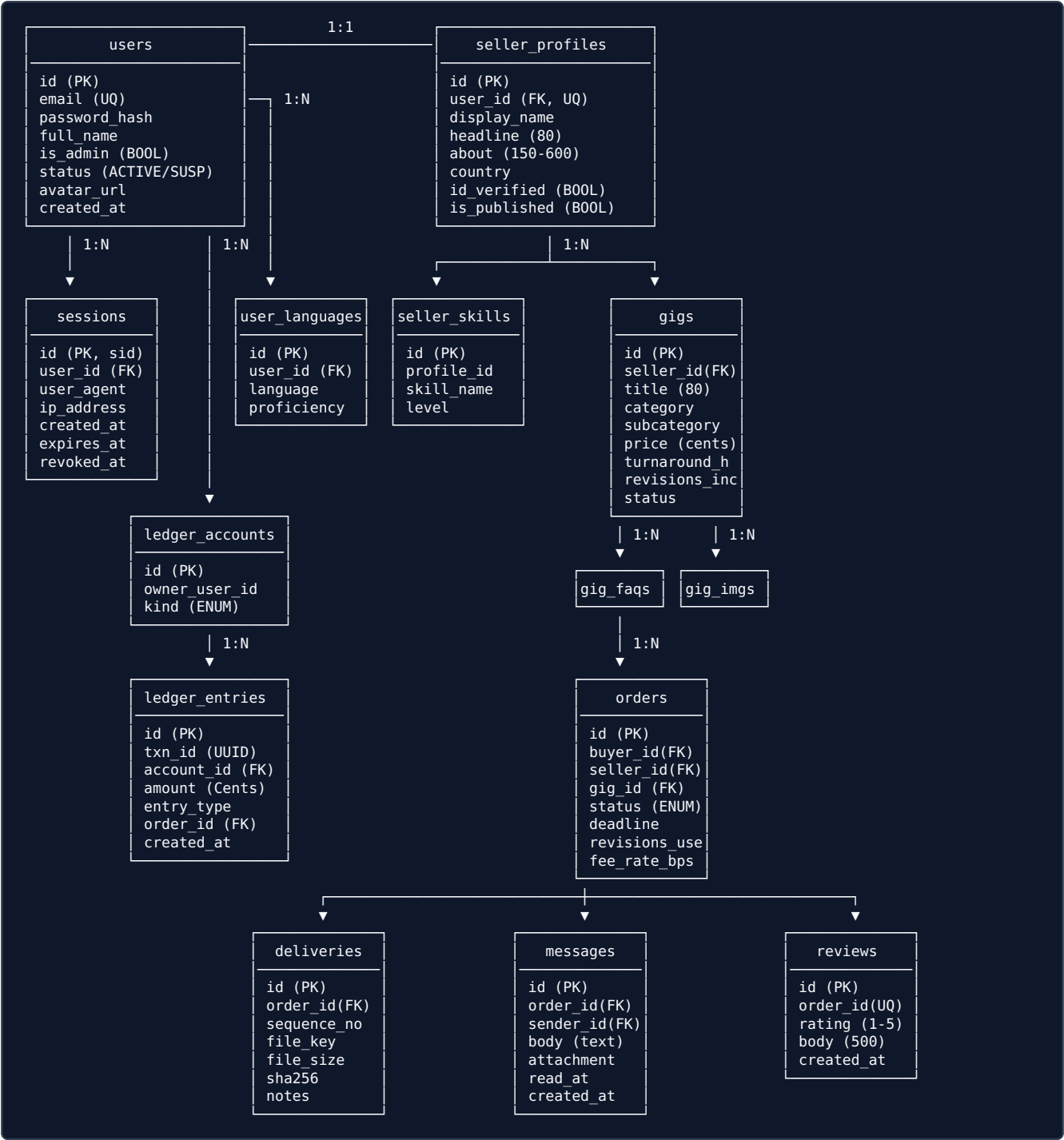
Altered Columns & Enums:

- `users`: Dropped `role` enum. Added `is_admin` (BOOLEAN DEFAULT FALSE) and `status` (ENUM: 'ACTIVE', 'SUSPENDED').
- `gigs`: Added `subcategory` (VARCHAR(60)), `revisions_included` (INT DEFAULT 2), and `status` (ENUM: 'DRAFT', 'PUBLISHED', 'PAUSED') replacing boolean `is_active`. Title character limit reduced to 80 chars.
- `orders`: Added `fee_rate_bps` (INT DEFAULT 2000, i.e., 20.00%), `revisions_used` (INT DEFAULT 0), `revision_deadline` (TIMESTAMPTZ NULL), and `disputed_at` (TIMESTAMPTZ NULL).
- `deliveries`: Renamed `revision_count` to `sequence_no`. Added `file_size` (BIGINT), `sha256` (CHAR(64)), and `file_tree` (JSONB NULL).
- `ledger_entries`: Replaced direct `user_id` with `account_id` (FK to `ledger_accounts`) and added `txn_id` (UUID) to group multi-leg balanced transactions.

Strategic Index Additions:

- `orders(status, deadline)`: Compound index for sub-millisecond evaluation of the `Late` tab and 10-minute reconciler queries.
- `messages(order_id, created_at)`: High-performance message pagination and 15s polling.
- `notifications(user_id, read_at)`: Instant unread badge aggregation.
- `favorites(gig_id)`: Fast aggregation of public gig favorite counters.

9.2 Entity Relationship Diagram (ERD)



10. Order State Machine (Revised)

10.1 Complete Transition Matrix

The order lifecycle is governed by an explicit state machine. Each state transition is guarded by pre-condition invariants, emits an immutable audit event (`order_events`), and triggers deterministic ledger side-effects:

Transition ID	Current State	Target State	Authorized Actor	Guard Conditions / Invariants	Ledger & System Side-Effects
T-01	None	PENDING_REQUIREMENTS	Buyer	Buyer wallet balance $\geq$ gig price.	Deducts price from Buyer <code>USER_AVAILABLE</code> , credits <code>ESCROW</code> . Inserts <code>ORDER_PLACED</code> event.
T-02	PENDING_REQUIREMENTS	IN_PROGRESS	Buyer	All mandatory requirement prompts answered.	Calculates <code>deadline = NOW() + INTERVAL (turnaround_hours)</code> . Inserts <code>REQUIREMENTS_SUBMITTED</code> event.
T-03	PENDING_REQUIREMENTS	CANCELLED	Buyer	Order unstated.	Deducts <code>ESCROW</code> , credits Buyer <code>USER_AVAILABLE</code> (100% refund). Status \$ o\$ <code>CANCELLED</code> .
T-04	IN_PROGRESS	DELIVERED	Seller	Deliverable uploaded to S3; valid <code>file_key</code> & notes.	Pauses order deadline clock. Calculates <code>auto_complete_at = NOW() + 72 hours</code> . Enqueues BullMQ auto-complete job.
T-05	DELIVERED	COMPLETED	Buyer	Explicit buyer approval click.	Cancels BullMQ job. Settles Escrow: debits <code>ESCROW</code> , credits Seller <code>USER_PENDING</code> ($P - F$), credits <code>PLATFORM_REVENUE</code> (F). Enqueues 3-day clearing job.
T-06	DELIVERED	COMPLETED	System	72 hours elapsed since delivery with zero buyer action.	Automatically triggered by BullMQ or 10-minute reconciler. Executes exact same financial settlement as T-05.
T-07	DELIVERED	IN_REVISION	Buyer	<code>revisions_used < revisions_included</code> . Feedback text ≥ 20 chars.	Cancels BullMQ auto-complete job. Increments <code>revisions_used</code> . Sets <code>revision_deadline = NOW() + 24 hours</code> . Stores feedback in <code>order_events.payload</code> .
T-08	IN_REVISION	DELIVERED	Seller	New deliverable uploaded.	Increments delivery <code>sequence_no</code> . Sets new <code>auto_complete_at = NOW() + 72 hours</code> . Re-enqueues BullMQ job.
T-09	DELIVERED or IN_PROGRESS	DISPUTED	Buyer or Seller	Valid order participant. Reason text ≥ 50 chars.	Pauses all auto-complete timers and deadlines. Flags order for admin arbitration. Sets <code>disputed_at = NOW()</code> .
T-10	DISPUTED	COMPLETED	Admin	Admin review complete: rules in favor of seller.	Executes standard financial settlement (T-05). Status \$ o\$ <code>COMPLETED</code> .
T-11	DISPUTED	CANCELLED	Admin	Admin review complete: rules in favor of buyer.	Full refund: debits <code>ESCROW</code> , credits Buyer <code>USER_AVAILABLE</code> . Status \$ o\$ <code>CANCELLED</code> .
T-12	DISPUTED	COMPLETED	Admin	Admin rules custom partial split.	Debits <code>ESCROW</code> , credits Buyer, Seller, and Platform according to admin-specified basis points.
T-13		CANCELLED	Buyer		100% full refund from <code>ESCROW</code> to Buyer

Transition ID	Current State	Target State	Authorized Actor	Guard Conditions / Invariants	Ledger & System Side-Effects
	IN_PROGRESS or IN_REVISION			NOW() > deadline + 24 hours (ORD-15 Late Remedy).	USER_AVAILABLE . Status \$ o\$ CANCELLED .
T-14	IN_PROGRESS	CANCELLED	Buyer & Seller	Mutual cancellation agreement reached.	100% full refund from ESCROW to Buyer USER_AVAILABLE . Status \$ o\$ CANCELLED .

10.2 Concurrency Safety: Optimistic Locking Pattern

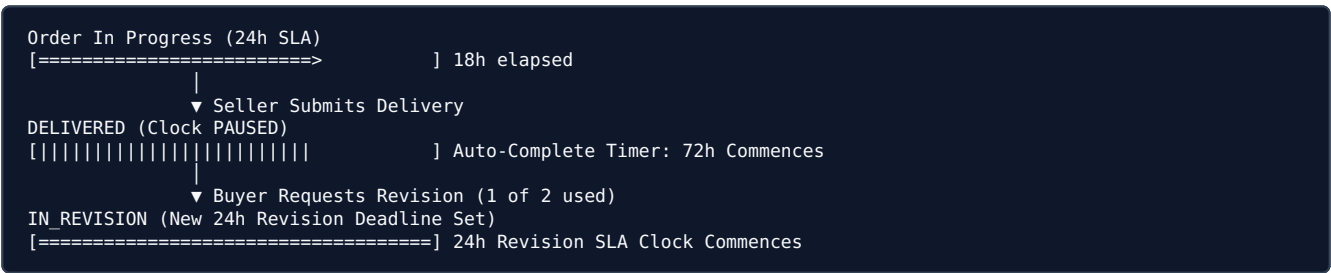
To prevent critical race conditions (e.g., a buyer clicking "Accept Delivery" at the exact second the 72-hour BullMQ background worker triggers auto-completion), all status updates strictly execute via atomic SQL statements with expected-state preconditions:

```
// Optimistic Concurrency Control in Prisma / PostgreSQL
async function transitionOrderStatus(
  orderId: string,
  expectedStatus: OrderStatus,
  newStatus: OrderStatus,
  tx: Prisma.TransactionClient
): Promise<Order> {
  const result = await tx.$executeRaw`
    UPDATE "orders"
    SET "status" = ${newStatus}::"OrderStatus",
        "updated_at" = NOW()
    WHERE "id" = ${orderId}
        AND "status" = ${expectedStatus}::"OrderStatus"
  `;

  if (result === 0) {
    throw new ConcurrencyConflictError(
      `Order ${orderId} state conflict: Expected ${expectedStatus}, but state was altered concurrently.`
    ); // Maps to HTTP 409 Conflict
  }

  return await tx.order.findUniqueOrThrow({ where: { id: orderId } });
}
```

10.3 Deadline Clock & Revision SLA Rules



- 1. **Primary Deadline:** Begins the instant the buyer submits requirements (IN_PROGRESS). Duration is fixed at either 24 or 48 hours according to gig parameters.
- 2. **Clock Pause:** The moment the seller submits a delivery, the primary delivery SLA clock is paused. The seller cannot be marked Late while awaiting buyer review.
- 3. **Revision Deadline:** If the buyer requests a revision within their allotted quota, the order transitions to IN_REVISION and a new, independent revision_deadline is set to NOW() + 24 hours .
- 4. **Late Calculation:** \$\$ ext{is_late} = (ext{NOW}() > ext{deadline} \wedge ext{status} = ext{IN_PROGRESS}) \vee (ext{NOW}() > ext{revision_deadline} \wedge ext{status} = ext{IN_REVISION}) \$\$

11. Double-Entry Escrow Ledger & Wallet Design

11.1 Analysis of Defects Remediated from v1.0

A rigorous forensic audit of the v1.0 financial implementation revealed five critical flaws that rendered the platform vulnerable to balance drift and data corruption:

1. **Broken Zero-Sum Invariant:** In v1.0, the `ESCROW_HOLD` transaction credited the escrow account with the gig price (+\$50), but failed to generate a corresponding debit leg against the buyer's balance. Consequently, the total sum of ledger entries after order creation was +\$50 rather than 0.
2. **Auto-Complete Escrow Leak:** In §16.3 of v1.0, the automated completion worker recorded only 2 ledger entries (crediting the seller and platform fee), omitting the crucial debit entry from escrow. Under auto-completion, the platform's escrow balance would have permanently inflated.
3. **Absence of a Formal Account Model:** Platform escrow and revenue had no dedicated database entity. Using `user_id = NULL` made it impossible to query system escrow vs. system revenue directly without ambiguous heuristic filters.
4. **Fragile Idempotency:** Idempotency relied solely on application-level status checks with no database-level unique constraint to reject duplicate settlement calls.
5. **Single-Bucket Balance Limitation:** Summing all user entries under `SUM(amount)` prevented separating immediately withdrawable funds from funds pending clearance—a core requirement of Fiverr's earnings dashboard.

11.2 Corrected Account Architecture

The persistence model introduces a dedicated `ledger_accounts` table:

```
CREATE TYPE "LedgerAccountKind" AS ENUM (  
  'USER_AVAILABLE',    -- Readily spendable or withdrawable balance  
  'USER_PENDING',      -- Earned seller funds held during 3-day clearing  
  'ESCROW',            -- Platform-held funds for active in-flight orders  
  'PLATFORM_REVENUE',  -- Retained platform commission (20%)  
  'BUYER_FUNDING'      -- External settlement source/sink (simulated banking rail)  
);  
  
CREATE TABLE "ledger_accounts" (  
  "id" UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  "owner_user_id" UUID REFERENCES "users"("id") ON DELETE RESTRICT,  
  "kind" "LedgerAccountKind" NOT NULL,  
  "created_at" TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  CONSTRAINT "uq_user_account_kind" UNIQUE ("owner_user_id", "kind")  
);
```

11.3 Double-Entry Event Specifications

Every business transaction is recorded as an immutable set of `ledger_entries` bound by a unique `txn_id`. In accordance with double-entry principles, **debits must exactly equal credits**:

$\sum (\text{Debits} + \text{Credits}) = 0$ (Zero-Sum Invariant)

Event 1: Virtual Balance Top-Up (\$50.00)

DEBIT (-)	CREDIT (+)
System BUYER_FUNDING: -\$50.00	Buyer USER_AVAILABLE: +\$50.00

Event 2: Order Placed & Escrow Locked (\$35.00 Order)

DEBIT (-)	CREDIT (+)
Buyer USER_AVAILABLE: -\$35.00	Platform ESCROW: +\$35.00

Event 3: Order Completed (20% Platform Fee = \$7.00)

DEBIT (-)	CREDIT (+)
Platform ESCROW: -\$35.00	Seller USER_PENDING: +\$28.00 Platform REVENUE: +\$7.00

Event 4: Fund Clearance (3 Days Post-Completion)

DEBIT (-)	CREDIT (+)
Seller USER_PENDING: -\$28.00	Seller USER_AVAILABLE: +\$28.00

Event 5: Order Cancelled / Refunded (100% Buyer Restitution)

DEBIT (-)	CREDIT (+)
Platform ESCROW: -\$35.00	Buyer USER_AVAILABLE: +\$35.00

Event 6: Simulated Seller Payout / Withdrawal (\$28.00)

DEBIT (-)	CREDIT (+)
Seller USER_AVAILABLE: -\$28.00	System BUYER_FUNDING: +\$28.00

11.4 Deferred Constraint Trigger (Mathematical Zero-Sum Invariant)

To guarantee that no transaction can ever commit with an unbalanced sum, PostgreSQL enforces a deferred constraint trigger:

```
CREATE OR REPLACE FUNCTION verify_ledger_transaction_zero_sum()
RETURNS TRIGGER AS $$
DECLARE
    v_sum BIGINT;
BEGIN
    SELECT COALESCE(SUM(amount), 0)
    INTO v_sum
    FROM "ledger_entries"
    WHERE "txn_id" = NEW."txn_id";

    IF v_sum <> 0 THEN
        RAISE EXCEPTION 'Ledger transaction % is unbalanced! Net sum is % cents (must be exactly 0)',
            NEW."txn_id", v_sum;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER "trg_assert_ledger_zero_sum"
AFTER INSERT OR UPDATE ON "ledger_entries"
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION verify_ledger_transaction_zero_sum();
```

11.5 Integer Cent Math, Fee Rounding & Database Idempotency

- 1. **Integer Representation:** All currency amounts are stored strictly as integer cents (\$1.00 = 100 cents). Float and decimal types are forbidden.
- 2. **Rounding Rule:** Platform fee is computed using **Round Half-Up** (`round_half_up(P * fee_rate_bps / 10000)`) arithmetic: $\$ \text{fee_cents} = \lfloor (\text{price_cents} \times \text{fee_rate_bps} + 5000) / 10000 \rfloor$ \$ For a \$35.00 order (3500 cents) at 2000 bps (20%): $\$ \text{\textit{\text{fee}}} = \lfloor (3500 \times 2000 + 5000) / 10000 \rfloor = 700 \text{\textit{\text{cents}}} (\$7.00)$ Seller portion is always strictly $\text{\textit{\text{seller_amount}}} = \text{\textit{\text{price_cents}}} - \text{\textit{\text{fee_cents}}}$.
- 3. **Database-Enforced Idempotency:** `sql -- Guarantees that settlement can never run twice for the same order CREATE UNIQUE INDEX "uq_order_ledger_settlement" ON "ledger_entries" ("order_id", "entry_type") WHERE "order_id" IS NOT NULL;`
- 4. **Immutability Enforcement:** `UPDATE` and `DELETE` privileges on `ledger_entries` are explicitly revoked from the application database role. Corrections require compensating double-entry reversal transactions.

11.6 Wallet Dashboard Specifications

Modeled directly on Fiverr's Earnings dashboard, the `/wallet` route renders four distinct headline metrics:

Metric Card	Computation Formula	UI Meaning
Available for Withdrawal	<code>SUM(amount)</code> from <code>USER_AVAILABLE</code> account	Immediately spendable on new orders or withdrawable.
Pending Clearance	<code>SUM(amount)</code> from <code>USER_PENDING</code> account	Earnings from completed orders undergoing 3-day holding.
In Active Orders	<code>SUM(price)</code> of orders where user is seller and status <code>€€ [IN_PROGRESS, DELIVERED, IN_REVISION]</code>	Expected revenue currently secured in platform escrow.
Earnings to Date	Cumulative credits to <code>USER_PENDING</code> over account lifetime	Lifetime gross earnings before withdrawals.

12. API Specification & Contract

All endpoints adhere to REST conventions, live under `/api/v1`, mandate JSON payloads validated by Zod, and require secure cookie authentication.

12.1 Authentication & Session Endpoints

Method	Endpoint	Access	Description	Request Payload / Response
POST	<code>/api/v1/auth/register</code>	Public	Register new base user	Body: <code>{ email, password, fullName }</code> Res: <code>201 Created { user }</code> (Sets auth cookie)
POST	<code>/api/v1/auth/login</code>	Public	Authenticate user	Body: <code>{ email, password }</code> Res: <code>200 OK { user }</code> (Sets auth & refresh cookies)
POST	<code>/api/v1/auth/logout</code>	Auth	Invalidate current session	Res: <code>200 OK</code> (Clears cookies, marks session revoked)
POST	<code>/api/v1/auth/logout-all</code>	Auth	Invalidate all sessions	Res: <code>200 OK</code> (Revokes all user session records)
GET	<code>/api/v1/me/sessions</code>	Auth	List active sessions	Res: <code>200 OK { sessions: [{ id, ip, ua, createdAt }] }</code>
DELETE	<code>/api/v1/me/sessions/:id</code>	Auth	Revoke specific session	Res: <code>200 OK</code>

12.2 Seller Profile & Onboarding Endpoints

Method	Endpoint	Access	Description	Request Payload / Response
GET	/api/v1/me/seller-profile	Auth	Get current user's profile	Res: 200 OK { profile, skills, languages }
PUT	/api/v1/me/seller-profile	Auth	Update seller profile	Body: { displayName, headline, about, country, languages, skills } Res: 200 OK
POST	/api/v1/me/seller-profile/submit	Auth	Submit ID verification stub	Body: { idDocumentType, documentReference } Res: 200 OK { status: "PENDING_VERIFICATION" }
GET	/api/v1/me/dashboard	Seller	Seller dashboard stats	Res: 200 OK { checklist, stats, walletSummary }

12.3 Catalog & Discovery Endpoints

Method	Endpoint	Access	Description	Request Payload / Response
GET	/api/v1/gigs	Public	Search and filter catalog	Query: ?category=&subcategory=&search=&sort=&page= Res: 200 OK { gigs: [...], total, page }
GET	/api/v1/gigs/:slug	Public	Get full gig details	Res: 200 OK { gig, sellerCard, faqs, images, reviews }
POST	/api/v1/gigs	Verified Seller	Create gig via 5-step wizard	Body: { title, category, subcategory, price, turnaroundHours, revisionsIncluded, tags, faqs, requirementsPrompt } Res: 201 Created { gig }
PUT	/api/v1/gigs/:id	Seller	Update gig details	Body: Partial gig schema Res: 200 OK
POST	/api/v1/gigs/:id/favorite	Auth	Toggle favorite heart	Res: 200 OK { isFavorited: true, count: 14 }
GET	/api/v1/gigs/:id/more-from-seller	Public	Seller's other gigs	Res: 200 OK { gigs: [...] } (Max 5)

12.4 Order Lifecycle & Workspace Endpoints

Method	Endpoint	Access	Description	Key Security & Idempotency Rules
POST	/api/v1/orders	Auth (Buyer)	Place order & lock escrow	Header: Idempotency-Key: <UUID> Body: { gigId } Error: 402 Payment Required if balance insufficient
GET	/api/v1/orders/:id	Buyer / Seller / Admin	Fetch order workspace	Security Fix: NEVER serializes raw file_url to buyers during DELIVERED . Returns only watermarkedPreviewUrl , fileName , fileSize , sha256 , and fileTree .
GET	/api/v1/orders/user/me	Auth	Manage Orders listing	Query: ?tab=priority\ active\ late\ delivered\ completed\ cancelled\ starred Res: 200 OK { counts: {...}, orders: [...] }
POST	/api/v1/orders/:id/requirements	Buyer	Submit requirements	Body: { answers: [...] } \$→\$ transitions to IN_PROGRESS
POST	/api/v1/orders/:id/complete	Buyer	Accept delivery	Header: Idempotency-Key: <UUID> Optimistic lock: status must be DELIVERED .
POST	/api/v1/orders/:id/revision	Buyer	Request revision	Body: { feedback } Error: 422 Unprocessable Entity if revisions_used >= revisions_included .
POST	/api/v1/orders/:id/cancel	Buyer	Cancel order	Allowed if PENDING_REQUIREMENTS OR NOW() > deadline + 24h (ORD-15).
POST	/api/v1/orders/:id/dispute	Buyer / Seller	Open dispute	Header: Idempotency-Key: <UUID> Body: { reason }
POST	/api/v1/orders/:id/star	Auth	Star / unstar order	Res: 200 OK { isStarred: true }

12.5 Delivery & Upload Endpoints

Method	Endpoint	Access	Description	Request Payload / Response
POST	/api/v1/deliveries/presign-upload	Assigned Seller	Request pre-signed S3 PUT URL	Body: { filename, contentType, fileSize } Res: 200 OK { uploadUrl, fileKey: "deliveries/{ordId}/{uuid}/{name}" }
POST	/api/v1/orders/:id/deliveries	Assigned Seller	Commit delivery record	Security Fix: Accepts fileKey (server-generated), NOT arbitrary client URLs. Body: { fileKey, notes } Res: 201 Created

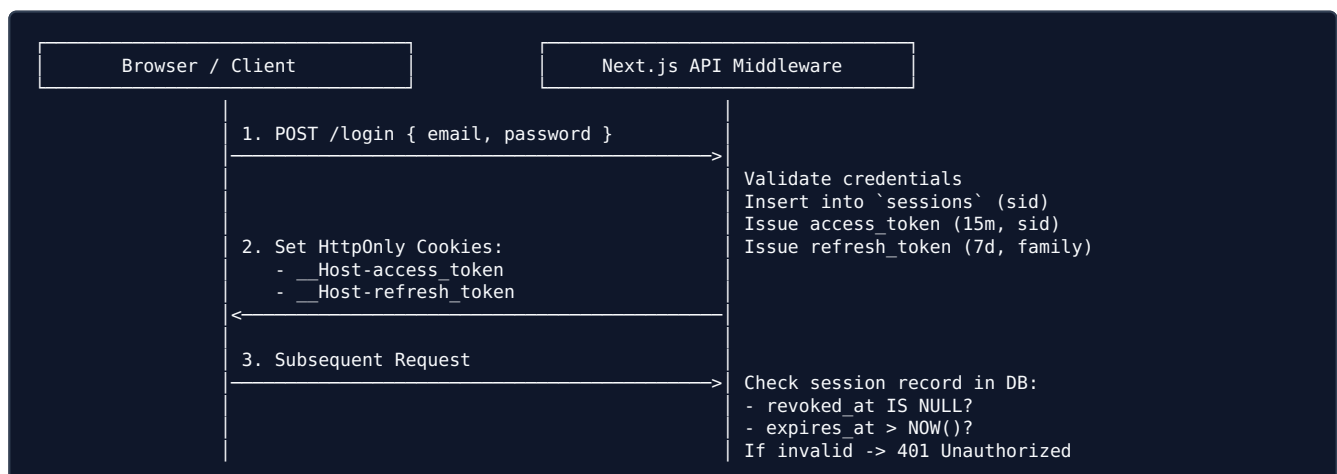
12.6 Messaging, Notifications & Reviews

Method	Endpoint	Access	Description	Request Payload / Response
GET	/api/v1/orders/:id/messages	Order Participant	Fetch message thread	Polled every 15s. Returns messages with read statuses.
POST	/api/v1/orders/:id/messages	Order Participant	Send message	Body: { body, attachmentKey } Warns if anti-leakage regex triggers.
GET	/api/v1/notifications	Auth	List user notifications	Res: 200 OK { unreadCount: 3, notifications: [...] }
POST	/api/v1/notifications/read	Auth	Mark all as read	Res: 200 OK
GET / PUT	/api/v1/notification-prefs	Auth	Get/update preferences	Body: Matrix of notification category flags.
POST	/api/v1/orders/:id/review	Buyer	Submit rating & review	Body: { rating: 1-5, body: text } Permitted only if status is COMPLETED .
GET	/api/v1/wallet	Auth	Get wallet summary	Res: 200 OK { available, pending, inActiveOrders, lifetimeEarnings }
GET	/api/v1/wallet/activity	Auth	Transaction history	Supports ?format=csv export.

13. Authentication, Authorization & Security Architecture

13.1 State-Backed Session Management & Refresh Token Rotation

In v1.0, token revocation was purely conceptual. Version 1.1 establishes a production-grade session layer:



- Session Table (sessions):** Tracks `id` (UUIDv4 `sid`), `user_id`, `user_agent`, `ip_address`, `created_at`, `expires_at`, and `revoked_at`.
- Access Token:** Short-lived (15 minutes), signed via EdDSA/HS256 with payload { sub: `userId`, sid: `sessionId`, isAdmin: `boolean` }.
- Refresh Token:** Long-lived (7 days), stored in a separate `__Host-refresh_token` HTTP-only cookie. Employs **Automatic Token Rotation (ATR)**. If a used refresh token is presented a second time (indicating token theft), the entire token family is immediately revoked.
- Instant Revocation ("Log Out Everywhere"):** Setting `revoked_at = NOW()` on the user's sessions immediately terminates access across all active browser instances on the next request.

13.2 Multi-Tier Rate Limiting & Abuse Prevention

Rate limiting is enforced at the edge/middleware using Upstash Redis sliding window counters:

Target Route / Action	Rate Limit Threshold	Window	Mitigation Behavior on Breach
Login (/auth/login)	10 requests	1 minute	Combined Key: ip + ":" + email . Returns 429 Too Many Requests . Progressive backoff: 5 failures locks account for 15 minutes.
Registration (/auth/register)	5 requests	1 hour	Key: ip . Prevents automated bot account harvesting.
Password Reset	3 requests	1 hour	Key: email . Mitigates enumeration and inbox spamming.
Pre-Signed Upload URL	20 requests	1 hour	Key: userId . Mitigates S3 storage flooding and cost exhaustion.
Order Messaging	30 messages	1 minute	Key: userId . Mitigates spamming order threads.
Public Catalog Search	120 requests	1 minute	Key: ip . Protects database search queries from denial of service.

13.3 Object-Level Authorization (IDOR Protection)

To prevent Insecure Direct Object Reference (IDOR) vulnerabilities, every private endpoint mandates contextual relationship verification:

```
// Middleware / Route Guard: Order Access Verification
export async function assertOrderParticipant(
  orderId: string,
  userId: string,
  isAdmin: boolean,
  tx: PrismaClient
): Promise<Order> {
  const order = await tx.order.findUnique({
    where: { id: orderId },
    include: { gig: true }
  });

  if (!order) {
    throw new NotFoundError("Order not found");
  }

  // Admin has global read/arbitration override
  if (isAdmin) return order;

  // Verify caller is strictly either the assigned buyer or seller
  if (order.buyerId !== userId && order.sellerId !== userId) {
    throw new ForbiddenError("You are not an authorized participant in this order workspace.");
  }

  return order;
}
```

13.4 Anti-Disintermediation & Contact Leakage Filter (MSG-03)

To safeguard platform escrow and protect users from off-platform fraud, all outgoing messages pass through an asynchronous pattern evaluation filter:

```
// Non-blocking anti-disintermediation pattern detection
const LEAKAGE_PATTERNS = [
  /[a-zA-Z0-9_%.+~]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi, // Emails
  /(\\+?\\d{1,3}[\\-\\.\\s]?\\(\\?\\d{3}\\)?[\\-\\.\\s]?\\d{3}[\\-\\.\\s]?\\d{4}/g, // Phone Numbers
  /(t\\.me|telegram\\.me|wa\\.me|whatsapp\\.com|paypal\\.me)/gi, // Direct messaging / payment apps
  /(zoom\\.us|meet\\.google\\.com|skype:)/gi // External video calls
];

export function detectContactLeakage(text: string): boolean {
  return LEAKAGE_PATTERNS.some(regex => regex.test(text));
}
```

Note: When a violation is detected, the UI prompts: "Keep payments and communication on the platform. Warning: Sharing contact details or taking payments outside Micro-Gig violates our Terms of Service and forfeits your escrow protections." The message is delivered, but the event is logged to `admin_audit` for risk profiling.

14. File Storage & Asset Protection Pipeline

14.1 S3 Pre-Signed Upload Security

To eliminate server bottlenecks and protect against malicious file injection: 1. **Server-Generated Object Keys:** The client never specifies the destination S3 key. The server generates: ``${text{key}} = \text{"deliveries/" + \text{orderId} + \text{"/" + \text{uuidv4()} + \text{"/" + \text{sanitize(filename)}}$`` 2. **Policy Condition Enforcement:** The pre-signed `PUT` policy enforces strict bounds: - `content-length-range`: ``${100, 52428800}`` bytes (100 B to 50 MB max). - `Content-Type`: Strictly matches verified MIME types (`application/zip`, `image/png`, `application/pdf`, etc.). 3. **Server-Side Verification:** When the seller calls `POST /api/v1/orders/:id/deliveries`, the server initiates an S3 `HeadObject` call to verify: - File exists at `fileKey`. - Reported size matches S3 byte count. - Initial 512 bytes are inspected for **magic-byte integrity** (e.g., verifying `PK` for ZIP files).

14.2 Watermarking Pipeline (DEL-06)

For image deliverables (`image/png`, `image/jpeg`, `image/webp`), an automated worker executes a non-destructive Sharp transformation:

```
import sharp from "sharp";

export async function generateWatermarkedPreview(
  inputBuffer: Buffer,
  orderNumber: string
): Promise<Buffer> {
  const svgWatermark = `
    <svg width="800" height="600" xmlns="http://www.w3.org/2000/svg">
      <text x="50%" y="50%" font-family="Inter, sans-serif" font-size="42"
        font-weight="bold" fill="rgba(255,255,255,0.4)"
        transform="rotate(-30 400 300)" text-anchor="middle">
        MICRO-GIG PREVIEW #${orderNumber} • UNACCEPTED DELIVERABLE
      </text>
    </svg>
  `;

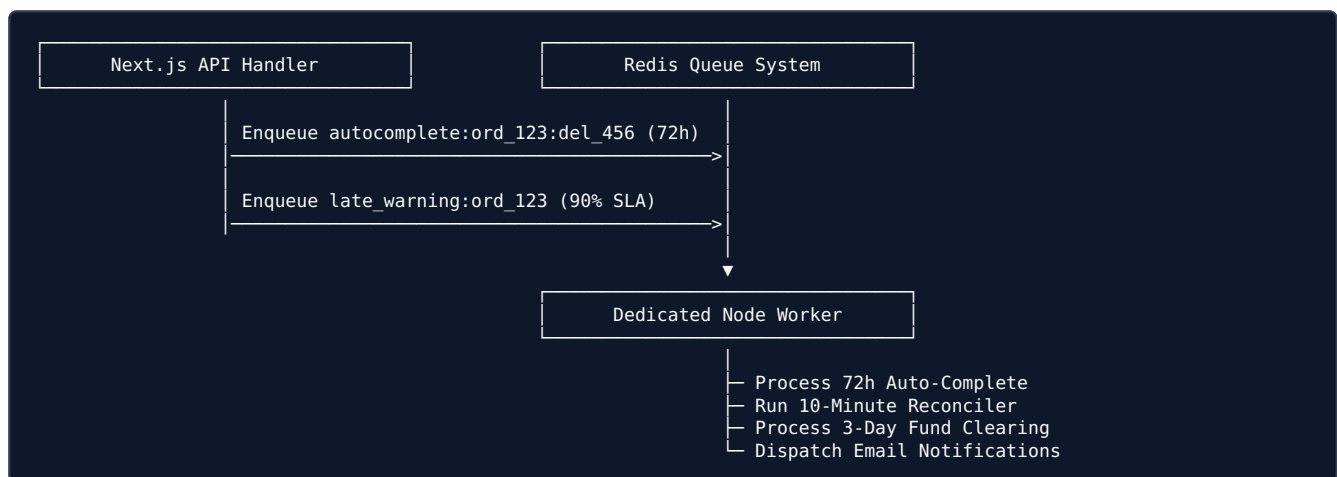
  return await sharp(inputBuffer)
    .resize(1200, 900, { fit: "inside", withoutEnlargement: true })
    .composite([
      { input: Buffer.from(svgWatermark), gravity: "center" }
    ])
    .webp({ quality: 80 })
    .toBuffer();
}
```

14.3 Non-Image Deliverable Protection (DEL-07)

For code files, ZIP packages, and text documents where visual watermarking is infeasible: - **Raw Object Lock:** Direct S3 download links are withheld until the order is `COMPLETED`. - **Pre-Completion Metadata:** The buyer receives: - Exact file name and byte size. - SHA-256 cryptographic checksum (permitting the buyer to verify file authenticity post-completion). - Sanitized directory file tree: For ZIP archives, the server reads the central directory header and renders an interactive folder tree (e.g., `src/index.js`, `styles/patch.css`, `README.md`) verifying that deliverable contents match project scope without exposing source code.

15. Background Job Processing & Worker System

The platform operates asynchronous background queues powered by **BullMQ** and **Redis**:



15.1 Deterministic Auto-Complete Scheduling & Reconciler

- Deterministic Job ID:** `jobId = "autocomplete:" + orderId + ":" + deliveryId`. If a buyer requests a revision, the application cancels `jobId`. When a new delivery is posted, a new job is cleanly scheduled without risk of duplicate timers.
- 10-Minute Reconciler Cron:** To ensure resilience against Redis crashes or dropped jobs, a cron job executes every 10 minutes: `sql -- Find any delivered order that exceeded 72 hours without completing`
`SELECT id, buyer_id, seller_id, price FROM "orders" WHERE "status" = 'DELIVERED' AND "auto_complete_at" IS NOT NULL AND "auto_complete_at" <= NOW();` For any returned records, the worker invokes `executeOrderCompletion(orderId, 'SYSTEM_RECONCILER')`, ensuring that escrow is released within ≤ 10 minutes of deadline expiry.

15.2 Seller Balance Clearing Worker (3 Days)

When an order reaches `COMPLETED`, seller earnings are deposited into `USER_PENDING`. A delayed BullMQ job scheduled for $T + 72$ hours (3 days) executes the transfer: `$$USER_PENDING → USER_AVAILABLE$$`. This holding period mirrors Fiverr's clearing buffer, providing a safety window against post-delivery payment disputes.

16. UI/UX Specifications & Wireframes

The user interface follows a modern, high-contrast, clean design system built on **Tailwind CSS** and **Inter** typography. - **Primary Accent:** Emerald Green (`#1DBF73`) — symbolizing trust, escrow security, and completion). - **Secondary / Text:** Charcoal Black (`#404145` and `#222325`). - **Surface / Backgrounds:** Pure

White (#FFFFFF) with Neutral Gray borders (#E4E5E7) and Soft Slate containers (#F7F7F7). - **Empty States:** Fiverr-inspired encouragement patterns (e.g., "Beginnings are so exciting! Create your first gig to start receiving orders.").

16.1 Screen 1: Marketplace Catalog (/gigs)

MicroGig Logo

Search gigs (e.g. CSS bug, logo, resume)...

(Bell)

Avatar v

Categories: [All] [Programming v] [Design v] [Writing v] [Video v] [Marketing v]

Subcategories: [All] > [CSS Bug Fixes] [Landing Page] [Automation Script]

Filter: [Sort By: Top Rated v] [Price: Under \$50] [Delivery: 24h Only] 124 Results

Thumbnail Image Avatar Alex Chen [V] I will fix 1 responsive CSS or layout bug ★ 4.9 (128) ♥ 34 From \$25.00	Thumbnail Image Avatar Sara Connor [V] I will design 1 modern vector app icon fast ★ 5.0 (42) ♥ 12 From \$35.00	Thumbnail Image Avatar Marcus Vance I will proofread 1000 words in 24 hours ★ 4.8 (89) ♥ 58 From \$15.00
--	---	--

16.2 Screen 2: 5-Step Gig Creation Wizard (/gigs/new)

MicroGig

Step 1: Overview > Step 2: Pricing > Step 3: Description > ...

● [o] [o] [o] [Save Draft]

GIG TITLE

[I willl] [fix one responsive layout or CSS bug in your web app] (58/80)

CATEGORY & SUBCATEGORY

[Programming & Tech v] [CSS & Bug Fixes v]

SEARCH TAGS (Press Enter to add, max 5)

[css x] [html x] [responsive x] [bug-fix x] [flexbox x]

TURNAROUND TIME

(●) 24 Hours () 48 Hours

REVISIONS INCLUDED

[2 Revisions Included v]

FIXED PRICE (\$5 - \$50)

[\$ 25.00]

[Cancel] [Continue to Next Step >]

16.3 Screen 3: Gig Details & Order Entry (/gigs/[slug])

< Back to Programming & Tech

I will fix one responsive layout or CSS bug in your web app

★ 4.9 (128 Reviews) • Category: Programming & Tech > CSS Bug Fixes

[Main Showcase Image / Video]

[Thumb 1] [Thumb 2] [Thumb 3]

About This Gig

Send me your broken CSS, flexbox, or mobile responsive layout snippet...

Frequently Asked Questions (FAQ)

[v] What frameworks do you support?
I work with Tailwind CSS, standard CSS modules, Bootstrap, and Styled Components.

[>] Do I need to provide full repository access?

About The Seller

[Avatar] Alex Chen [Verified] • Full-Stack Web Specialist

From: Canada • Member Since: Jan 2024 • Avg Response: 1 Hour

Last Delivery: 3 Hours Ago • Order Completion Rate: 99%

More From This Seller

[Card: I will build 1 landing page \$50]

[Card: I will integrate 1 webhook API \$30]

Fixed Price Service \$25.00

• 24 Hours Fast Delivery

• 2 Revisions Included

• Escrow Payment Protection

Requirements Needed:

• Description of bug & live URL/repo

[Order Now & Lock Escrow (\$25.00)]

Protected by Micro-Gig Double-Entry Escrow

16.4 Screen 4: Order Workspace (/orders/[id])

Order #ORD-84920 • Placed on Sep 30, 2026 • Total: \$25.00 (Escrow Secured)

Status: [DELIVERED] Countdown: [71h : 42m Left]

Stepper: (✓) Placed --> (✓) Requirements --> (●) Delivered --> () Completed

[Details]

[Deliverables (1)]

[Messages & Files (3)]

[Activity Audit]

DELIVERABLE SUBMITTED BY SELLER (Uploaded 20 minutes ago)

Note: "Fixed the mobile header overlay issue and updated the media queries."

[ZIP] css-patch-v1.zip (1.2 MB)

SHA-256: 4a8f9c2d1e0b5a3f...

Contents Preview:

└─ styles/navigation.css (modified)

└─ README-patch-instructions.txt

(!) Raw download unlocks upon completion. Review the notes & file tree above.

Buyer Actions:

[Accept Delivery & Release Funds]

[Request Revision (1 of 2 Revisions Used)]

[Need Help? Open a Dispute]

CONFIDENTIAL • CORE ENGINEERING TEAM • SEPTEMBER 2026

Page 31 of 44

16.5 Screen 5: Manage Orders (/orders)

+-----+ [Switch View: Buyer / Seller v]					
+-----+ Manage Orders					
+-----+ Tabs: [Priority (2)] [Active (4)] [Late (1)] [Delivered (1)] [Completed (45)] [Cancel]					
+-----+ Search orders...					
+-----+					
+-----+ ORDER GIG TITLE BUYER/SELLER DUE DATE TOTAL STATUS					
+-----+ #84920 Fix responsive CSS bug Alex Chen In 22 Hours \$25.00 DELIV					
+-----+ #84918 Vector app icon design Sara Connor YESTERDAY! \$35.00 [LATE]					
+-----+ (!) 24h past deadline: Buyer Cancel Remedy Available					
+-----+ #84890 Proofread 1000 words John Miller Sep 28 \$15.00 COMPL					
+-----+					

16.6 Screen 6: Seller Dashboard & Onboarding (/seller/dashboard)

+-----+ Seller Dashboard					Status: [Level 0]
+-----+					
+-----+ ONBOARDING CHECKLIST (Profile Strength: 75%)					
+-----+ [✓] Complete Profile Bio, Languages & Skills					
+-----+ [✓] Submit ID Verification (Approved - "Verified" Badge Active)					
+-----+ [✓] Create Your First Micro-Gig					
+-----+ [●] Publish Gig to Marketplace [Publish Now Button]					
+-----+					
+-----+ HEADLINE PERFORMANCE STATS					
+-----+					
+-----+ Net Earnings Avg Selling Price On-Time Delivery Completion Rate					
+-----+ \$ 640.00 \$ 32.50 98.2% 100%					
+-----+					
+-----+					

16.7 Screen 7: Wallet & Earnings View (/wallet)

+-----+ Financial Wallet & Ledger					
+-----+					
+-----+ +-----+ +-----+ +-----+ +-----+					
+-----+ Available Funds Pending Clearance In Active Orders Lifetime Earnings					
+-----+ \$ 180.00 \$ 56.00 (3d hold) \$ 70.00 (Escrow) \$ 876.00					
+-----+					
+-----+ [Withdraw Balance] [Add Virtual Test Funds]					
+-----+					
+-----+ TRANSACTION ACTIVITY [Export CSV Report v]					
+-----+ DATE TXN ID DESCRIPTION TYPE AMOUNT NET BAL					
+-----+ Sep 30, 2026 txn_8921 Order #84920 Escrow Release ORDER_COMPL +\$20.00 \$180.00					
+-----+ Sep 29, 2026 txn_8874 Virtual Wallet Top-Up TOP_UP +\$50.00 \$160.00					
+-----+ Sep 28, 2026 txn_8842 Order #84890 Escrow Lock ORDER_PLACED -\$15.00 \$110.00					
+-----+					

17. Non-Functional Requirements & Operational SLAs

Category	Metric / Constraint	Target SLA	Verification Method
Performance	Largest Contentful Paint (LCP)	$\leq 2.5 \text{ ext\{seconds\}}\$$	Google Lighthouse / Vercel Web Analytics
Performance	API Response Latency (p95)	$\leq 500 \text{ ext\{ms\}}\$$	Structured server log telemetry
Availability	Web App & API Uptime	99.5%\\$	External Pingdom / BetterUptime health check monitor
Financial Integrity	Ledger Net Balance Balance	Exactly \\$0.00\\$ Drift	Nightly automated SQL assertion job: verifies $\sum \text{ext\{amount\}} = 0\$$ for all <code>txn_id</code> and $\sum \text{ext\{ESCROW\}} = \sum(\text{ext\{open orders\}})\$$
Automation SLA	Auto-Completion Job Delay	$\leq 10 \text{ ext\{minutes\}}\$$ drift	BullMQ scheduler backed by 10-minute fallback reconciler cron
Messaging Latency	Message Dispatch & Sync	$\leq 20 \text{ ext\{seconds\}}\$$	Client 15s polling cadence with immediate optimistic local rendering
Accessibility	Color Contrast & Keyboard Nav	WCAG 2.1 AA	Axe Core automated accessibility test suite
Data Privacy	Anonymized Account Deletion	GDPR / CCPA Compliant	Account deletion anonymizes PII (<code>full_name</code> , <code>email</code> , <code>avatar_url</code>) while preserving immutable ledger rows with <code>ON DELETE RESTRICT</code>

18. Team Structure & Re-Scoped Sprint Plan

18.1 Engineering Role Distribution (Team of 4)

To resolve the defect in v1.0 where Member 2 experienced idle periods in Sprint 1 and Sprint 3 was overloaded, team allocations are restructured:

Member 1: Frontend & UI Lead - Catalog & category navigation - Gig detail & checkout view - Responsive layout & design sys - Manage Orders 7-tab dashboard	Member 2: Delivery & Media Lead 5-Step Gig Creation Wizard - S3 upload & preview pipeline - Gallery carousel & FAQ UI - Order workspace delivery form
Member 3: Backend & Ledger Lead - Postgres schema & Prisma ORM - State machine & optimistic lk - Double-entry ledger & triggers - Auth, sessions & IDOR guards	Member 4: DevOps & Platform - Redis & BullMQ infrastructure 10m reconciler & clearing cron - Docker Compose & S3 buckets - Seed data & CI/CD deployment

18.2 Re-Scoped Sprint Schedule (6 Weeks Total)

Sprint 1 (Weeks 1-2): Architecture Foundations & Catalog

- Member 1 (Frontend):** Design system tokens (Tailwind), global navigation, catalog search/filter layout, and category/subcategory hierarchy.
- Member 2 (Delivery & Media):** 5-step gig creation wizard client UI, gallery multi-image upload component, and gig FAQ accordion manager.
- Member 3 (Backend & Ledger):** PostgreSQL database schema, Prisma migrations, authentication API with session table, and seller onboarding endpoints.

- **Member 4 (DevOps):** Docker Compose development environment, S3/MinIO bucket provisioning, CI lint/typecheck workflows, and comprehensive seed data script.
- **Milestone 1:** Authenticated user can onboard as a seller, build a gig through the wizard, and view it in the public catalog.

Sprint 2 (Weeks 3-4): Order Engine, State Machine & Financial Ledger

- **Member 1 (Frontend):** Order workspace UI (`/orders/[id]`), status stepper, countdown component, and Manage Orders 7-tab dashboard with live counts.
- **Member 2 (Delivery & Media):** Requirements submission form, revision request interface, and order-scoped async messaging thread UI.
- **Member 3 (Backend & Ledger):** Order state machine implementation with optimistic locking, double-entry escrow ledger with deferred trigger, and Manage Orders API with Late flag.
- **Member 4 (DevOps):** BullMQ delayed queue integration, in-app notification bell dispatcher, and transactional email provider integration.
- **Milestone 2:** Buyer can place an order, lock escrow, submit requirements, view the workspace countdown, and send async messages.

Sprint 3 (Weeks 5-6): Delivery Pipeline, Trust, Hardening & Staging Deploy

- **Member 1 (Frontend):** Wallet and Earnings dashboard (`/wallet`), review modal, favorites heart toggle, and Admin arbitration screens.
- **Member 2 (Delivery & Media):** S3 pre-signed upload handler, delivery review panel, non-image file tree reader, and image watermarking worker.
- **Member 3 (Backend & Ledger):** 24h late delivery cancellation remedy, dispute resolution API, admin audit logging, and automated IDOR penetration tests.
- **Member 4 (DevOps):** 10-minute fallback reconciler cron, 3-day fund clearing job, staging deployment on Vercel/Supabase, and load testing.
- **Milestone 3:** Full end-to-end lifecycle verified (Order \$ o\$ Deliver \$ o\$ Watermark \$ o\$ Accept / Auto-Resolve \$ o\$ Ledger Settle \$ o\$ Review).

18.3 Graceful Feature Drop Order

If unexpected technical roadblocks compress development time, features must be dropped strictly in the following priority order:

\$\$ ext{Favorites} → ext{"More from Seller"} → ext{Review Reminders} → ext{Regex Message Warnings} → ext{CSV Export} → ext{Session List UI}\$\$

CRITICAL NON-NEGOTIABLES (NEVER CUT): 1. **Ledger Correctness:** Zero-sum double-entry transactions and deferred trigger constraints. 2. **Object-Level IDOR Security:** Route guards preventing cross-account order or deliverable access. 3. **10-Minute Reconciler:** Background safety net ensuring no escrow funds are ever stranded.

19. Risk Assessment & Open Decisions

#	Identified Risk / Decision Item	Severity	Resolution / Architectural Mitigation
1	Scope Creep from Messaging & Reviews	High	Resolution: Implement minimal "MVP-Lite" specifications. Messaging uses 15-second polling (zero WebSockets). Reviews are restricted to single 1–5 star ratings after completion.
2	Seller vs. Buyer Platform Fee Distribution	Medium	Decision: Keep the platform fee strictly 20% on the seller side for MVP. Buyer checkout displays a flat, zero-fee total (\$25.00 order costs exactly \$25.00). Dual-sided fees are deferred to Phase 2.
3	Virtual Balance Friction for Buyers	Low	Mitigation: When a buyer attempts checkout with insufficient balance, the UI provides a prominent "Add \$50 Virtual Test Funds" button directly in the modal, allowing immediate checkout testing.
4	3-Day Seller Clearing Period Delay	Medium	Decision: Retain the 3-day clearing period in the schema and worker logic. For local developer test suites, expose an environment toggle <code>SKIP_CLEARING_PERIOD=true</code> to instantly mature pending funds.
5	Manual Identity Verification Bottleneck	Medium	Mitigation: Admin panel includes a dedicated <code>/admin/verifications</code> queue where an admin can approve or reject seller ID stubs with a single click. Suitable for initial platform volumes (< 500 users).
6	Transactional Email Deliverability Failures	Medium	Mitigation: Enforce RFC 5322 domain syntax validation at signup; use a verified sending domain with DKIM/SPF on Resend or SendGrid; in-app notification bell ensures order visibility regardless of email delivery.
7	Redis Timer Eviction or Queue Failure	High	Mitigation: Completely mitigated by the 10-minute fallback reconciler cron (\$15.1), which queries PostgreSQL directly for any expired auto-complete orders.
8	Feature Bloat from Fiverr's Breadth	High	Mitigation: Rigid adherence to §2. Any capability not explicitly tagged "MVP" or "MVP-Lite" is strictly barred from Sprint 1–3 pull requests.

20. Glossary

- **Double-Entry Escrow:** An accounting system where every currency transfer consists of matching debit and credit entries whose algebraic sum equals zero.
- **Gig:** A standardized digital service offered by a freelancer with a fixed scope, fixed price (under \$50), and strict turnaround time (24–48 hours).
- **IDOR (Insecure Direct Object Reference):** A security flaw where an attacker can access private resources (orders, messages, files) simply by modifying an ID parameter in an API request.
- **Modular Monolith:** An architectural pattern where all code executes within a single deployable application but is strictly separated into distinct internal domain modules.
- **Optimistic Concurrency Control:** A concurrency management technique that verifies a database record has not been altered by another process before applying state changes.
- **Reconciler:** A scheduled cron process that compares expected system state in the database against worker queues to repair dropped or delayed asynchronous jobs.
- **SaaP (Service-as-a-Product):** Packaging freelance professional services as discrete, fixed-price items with standardized deliverables and zero scope negotiation.

21. Appendices

Appendix A: Environment Variables

```
# =====
# MICRO-GIG MARKETPLACE: ENVIRONMENT CONFIGURATION (.env.example)
# =====

# Application & Host
NODE_ENV="development"
PORT=3000
NEXT_PUBLIC_APP_URL="http://localhost:3000"

# PostgreSQL Database (Supabase / Neon / Local Docker)
DATABASE_URL="postgresql://postgres:postgres_secure_pass@localhost:5432/microgig?schema=public&connection_limit=20"
DIRECT_URL="postgresql://postgres:postgres_secure_pass@localhost:5432/microgig?schema=public"

# Redis & BullMQ Worker
REDIS_URL="redis://:redis_secure_pass@localhost:6379"

# Authentication & Session Security
JWT_ACCESS_SECRET="generate_with_openssl_rand_hex_64_character_minimum_secret_key"
JWT_REFRESH_SECRET="generate_with_openssl_rand_hex_64_character_minimum_refresh_key"
COOKIE_SECRET="generate_with_openssl_rand_hex_32_character_cookie_signing_secret"

# AWS S3 / Cloudflare R2 Cloud Storage
S3_REGION="auto"
S3_ENDPOINT="http://localhost:9000" # Local MinIO or https://<account>.r2.cloudflarestorage.com
S3_ACCESS_KEY_ID="minioadmin"
S3_SECRET_ACCESS_KEY="minioadmin"
S3_PUBLIC_BUCKET_NAME="microgig-showcase-public"
S3_PRIVATE_BUCKET_NAME="microgig-deliverables-private"

# Transactional Email (Resend / SendGrid)
EMAIL_FROM="Micro-Gig Platform <notifications@microgig.dev>"
RESEND_API_KEY="re_mock_development_api_key_or_live_key"

# Business & Financial Constants
PLATFORM_FEE_BPS=2000           # 20.00% Platform Commission
AUTO_COMPLETE_DELAY_HOURS=72    # 72 Hours Buyer Inactivity Window
REVISION_DEADLINE_HOURS=24      # 24 Hours Seller Revision SLA
CLEARING_PERIOD_DAYS=3          # 3 Days Seller Balance Hold Window
SKIP_CLEARING_PERIOD="false"    # Set to "true" for automated local test suites

# Feature Flags
ENABLE_DISINTERMEDIATION_WARNING="true"
ENABLE_CLAMAV_SCAN="false"
```

Appendix B: Complete Prisma Schema (Reference)

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  directUrl = env("DIRECT_URL")
}

generator client {
  provider = "prisma-client-js"
}

// -----
// ENUMS
// -----

enum UserStatus {
  ACTIVE
  SUSPENDED
}

enum GigStatus {
  DRAFT
  PUBLISHED
  PAUSED
}

enum OrderStatus {
  PENDING_REQUIREMENTS
  IN_PROGRESS
  DELIVERED
  IN_REVISION
  COMPLETED
  CANCELLED
  DISPUTED
}

enum DisputeStatus {
  OPEN
  RESOLVED_BUYER_REFUND
  RESOLVED_SELLER_RELEASE
  RESOLVED_SPLIT
}

enum LedgerAccountKind {
  USER_AVAILABLE
  USER_PENDING
  ESCROW
  PLATFORM_REVENUE
  BUYER_FUNDING
}

enum LedgerEntryType {
  TOP_UP
  ORDER_PLACED
  ORDER_COMPLETED
  CLEARING
  ORDER_REFUNDED
  WITHDRAWAL
  DISPUTE_ADJUSTMENT
}

enum NotificationType {
  ORDER_PLACED
  REQUIREMENTS_SUBMITTED
  DELIVERABLE_UPLOADED
  REVISION_REQUESTED
  ORDER_COMPLETED
  AUTO_COMPLETE_WARNING
  LATE_WARNING
  DISPUTE_OPENED
  NEW_MESSAGE
}

enum LanguageProficiency {
  BASIC
  CONVERSATIONAL
  FLUENT
  NATIVE
}

enum SkillLevel {
  BEGINNER
  INTERMEDIATE
  EXPERT
}
```

```
// -----
// IDENTITY, SESSIONS & PROFILES
// -----

model User {
  id          String          @id @default(uuid()) @db.Uuid
  email       String          @unique @db.VarChar(255)
  passwordHash String        @map("password_hash") @db.VarChar(255)
  fullName    String          @map("full_name") @db.VarChar(120)
  isAdmin     Boolean         @default(false) @map("is_admin")
  status      UserStatus      @default(ACTIVE)
  avatarUrl   String?         @map("avatar_url") @db.VarChar(512)
  createdAt   DateTime        @default(now()) @map("created_at") @db.Timestamptz
  updatedAt   DateTime        @updatedAt @map("updated_at") @db.Timestamptz

  sessions    Session[]
  sellerProfile SellerProfile?
  languages    UserLanguage[]
  buyerOrders Order[]        @relation("BuyerOrders")
  sellerOrders Order[]        @relation("SellerOrders")
  favorites    Favorite[]
  orderStars   OrderStar[]
  messages     Message[]
  notifications Notification[]
  notificationPref NotificationPref?
  ledgerAccounts LedgerAccount[]

  @@map("users")
}

model Session {
  id          String          @id @default(uuid()) @db.Uuid
  userId      String          @map("user_id") @db.Uuid
  userAgent   String?         @map("user_agent") @db.VarChar(512)
  ipAddress   String?         @map("ip_address") @db.VarChar(64)
  createdAt   DateTime        @default(now()) @map("created_at") @db.Timestamptz
  expiresAt   DateTime        @map("expires_at") @db.Timestamptz
  revokedAt   DateTime?       @map("revoked_at") @db.Timestamptz

  user        User           @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId, revokedAt])
  @@map("sessions")
}

model SellerProfile {
  id          String          @id @default(uuid()) @db.Uuid
  userId      String          @unique @map("user_id") @db.Uuid
  displayName  String          @map("display_name") @db.VarChar(80)
  headline    String          @db.VarChar(80)
  about       String          @db.VarChar(600)
  country     String          @db.VarChar(60)
  idVerified  Boolean         @default(false) @map("id_verified")
  isPublished Boolean         @default(false) @map("is_published")
  createdAt   DateTime        @default(now()) @map("created_at") @db.Timestamptz
  updatedAt   DateTime        @updatedAt @map("updated_at") @db.Timestamptz

  user        User           @relation(fields: [userId], references: [id], onDelete: Restrict)
  skills      SellerSkill[]
  gigs        Gig[]

  @@map("seller_profiles")
}

model SellerSkill {
  id          String          @id @default(uuid()) @db.Uuid
  profileId   String          @map("profile_id") @db.Uuid
  skillName   String          @map("skill_name") @db.VarChar(60)
  level       SkillLevel      @default(INTERMEDIATE)

  profile     SellerProfile @relation(fields: [profileId], references: [id], onDelete: Cascade)

  @@unique([profileId, skillName])
  @@map("seller_skills")
}

model UserLanguage {
  id          String          @id @default(uuid()) @db.Uuid
  userId      String          @map("user_id") @db.Uuid
  language    String          @db.VarChar(60)
  proficiency  LanguageProficiency @default(CONVERSATIONAL)

  user        User           @relation(fields: [userId], references: [id], onDelete: Cascade)
}
```

```

    @@unique([userId, language])
    @@map("user_languages")
}

// -----
// GIG CATALOG
// -----

model Gig {
    id                String      @id @default(uuid()) @db.Uuid
    sellerProfileId   String      @map("seller_profile_id") @db.Uuid
    slug              String      @unique @db.VarChar(120)
    title             String      @db.VarChar(80)
    category           String      @db.VarChar(60)
    subcategory        String      @db.VarChar(60)
    description        String      @db.Text
    priceCents         Int         @map("price_cents") // Strictly 500 to 5000 ($5 to $50)
    turnaroundHours    Int         @map("turnaround_hours") // Strictly 24 or 48
    revisionsIncluded  Int         @default(2) @map("revisions_included")
    requirementsPrompt String      @map("requirements_prompt") @db.Text
    status             GigStatus   @default(DRAFT)
    createdAt          DateTime    @default(now()) @map("created_at") @db.Timestamptz
    updatedAt          DateTime    @updatedAt @map("updated_at") @db.Timestamptz

    sellerProfile      SellerProfile @relation(fields: [sellerProfileId], references: [id], onDelete: Restrict)
    images              GigImage[]
    faqs                GigFaq[]
    tags                GigTag[]
    favorites            Favorite[]
    orders              Order[]

    @@index([category, subcategory, status])
    @@index([sellerProfileId])
    @@map("gigs")
}

model GigImage {
    id                String      @id @default(uuid()) @db.Uuid
    gigId             String      @map("gig_id") @db.Uuid
    imageUrl           String      @map("image_url") @db.VarChar(512)
    isPrimary          Boolean     @default(false) @map("is_primary")
    displayOrder       Int         @default(0) @map("display_order")

    gig               Gig         @relation(fields: [gigId], references: [id], onDelete: Cascade)

    @@index([gigId, displayOrder])
    @@map("gig_images")
}

model GigFaq {
    id                String      @id @default(uuid()) @db.Uuid
    gigId             String      @map("gig_id") @db.Uuid
    question           String      @db.VarChar(150)
    answer             String      @db.VarChar(600)

    gig               Gig         @relation(fields: [gigId], references: [id], onDelete: Cascade)

    @@index([gigId])
    @@map("gig_faqs")
}

model GigTag {
    id                String      @id @default(uuid()) @db.Uuid
    gigId             String      @map("gig_id") @db.Uuid
    tagName            String      @map("tag_name") @db.VarChar(30)

    gig               Gig         @relation(fields: [gigId], references: [id], onDelete: Cascade)

    @@unique([gigId, tagName])
    @@index([tagName])
    @@map("gig_tags")
}

model Favorite {
    id                String      @id @default(uuid()) @db.Uuid
    userId            String      @map("user_id") @db.Uuid
    gigId             String      @map("gig_id") @db.Uuid
    createdAt          DateTime    @default(now()) @map("created_at") @db.Timestamptz

    user              User        @relation(fields: [userId], references: [id], onDelete: Cascade)
    gig               Gig         @relation(fields: [gigId], references: [id], onDelete: Cascade)

    @@unique([userId, gigId])
    @@index([gigId])
    @@map("favorites")
}

```

```

}

// -----
// ORDERS & LIFECYCLE
// -----

model Order {
    id                String          @id @default(uuid()) @db.Uuid
    orderNumber       Int             @unique @default(autoincrement()) @map("order_number")
    buyerId           String          @map("buyer_id") @db.Uuid
    sellerId          String          @map("seller_id") @db.Uuid
    gigId             String          @map("gig_id") @db.Uuid
    status            OrderStatus     @default(PENDING_REQUIREMENTS)
    priceCents        Int             @map("price_cents")
    feeRateBps        Int             @default(2000) @map("fee_rate_bps")
    revisionsIncluded Int             @default(2) @map("revisions_included")
    revisionsUsed      Int             @default(0) @map("revisions_used")
    requirementsText   String?        @map("requirements_text") @db.Text
    deadline          DateTime?       @db.Timestamptz
    revisionDeadline   DateTime?       @map("revision_deadline") @db.Timestamptz
    autoCompleteAt     DateTime?       @map("auto_complete_at") @db.Timestamptz
    disputedAt        DateTime?       @map("disputed_at") @db.Timestamptz
    createdAt          DateTime       @default(now()) @map("created_at") @db.Timestamptz
    updatedAt          DateTime       @updatedAt @map("updated_at") @db.Timestamptz

    buyer             User            @relation("BuyerOrders", fields: [buyerId], references: [id], onDelete: Restrict)
    seller            User            @relation("SellerOrders", fields: [sellerId], references: [id], onDelete: Restrict)
    gig               Gig             @relation(fields: [gigId], references: [id], onDelete: Restrict)

    deliveries        Delivery[]
    messages          Message[]
    events            OrderEvent[]
    review            Review?
    dispute            Dispute?
    stars             OrderStar[]
    ledgerEntries     LedgerEntry[]

    @@index([buyerId, status])
    @@index([sellerId, status])
    @@index([status, deadline])
    @@index([status, autoCompleteAt])
    @@map("orders")
}

model OrderStar {
    id                String          @id @default(uuid()) @db.Uuid
    orderId           String          @map("order_id") @db.Uuid
    userId            String          @map("user_id") @db.Uuid
    createdAt         DateTime       @default(now()) @map("created_at") @db.Timestamptz

    order             Order           @relation(fields: [orderId], references: [id], onDelete: Cascade)
    user              User            @relation(fields: [userId], references: [id], onDelete: Cascade)

    @@unique([orderId, userId])
    @@map("order_stars")
}

model OrderEvent {
    id                String          @id @default(uuid()) @db.Uuid
    orderId           String          @map("order_id") @db.Uuid
    actorId           String?         @map("actor_id") @db.Uuid
    fromStatus        OrderStatus?    @map("from_status")
    toStatus          OrderStatus     @map("to_status")
    reason            String          @db.VarChar(255)
    payload           Json?
    createdAt         DateTime       @default(now()) @map("created_at") @db.Timestamptz

    order             Order           @relation(fields: [orderId], references: [id], onDelete: Cascade)

    @@index([orderId, createdAt])
    @@map("order_events")
}

model Delivery {
    id                String          @id @default(uuid()) @db.Uuid
    orderId           String          @map("order_id") @db.Uuid
    sequenceNo        Int             @default(1) @map("sequence_no")
    fileKey           String          @map("file_key") @db.VarChar(512)
    fileSize          BigInt          @map("file_size")
    sha256            String          @db.Char(64)
    notes             String          @db.Text
    fileTree          Json?           @map("file_tree")
    previewUrl        String?         @map("preview_url") @db.VarChar(512)
    createdAt         DateTime       @default(now()) @map("created_at") @db.Timestamptz

```

```

order      Order      @relation(fields: [orderId], references: [id], onDelete: Cascade)

@@index([orderId, sequenceNo])
@@map("deliveries")
}

model Dispute {
  id          String      @id @default(uuid()) @db.Uuid
  orderId     String      @unique @map("order_id") @db.Uuid
  openedById  String      @map("opened_by_id") @db.Uuid
  reason      String      @db.Text
  status      DisputeStatus @default(OPEN)
  adminNotes  String?     @map("admin_notes") @db.Text
  resolvedAt  DateTime?   @map("resolved_at") @db.Timestamptz
  createdAt   DateTime     @default(now()) @map("created_at") @db.Timestamptz

  order      Order      @relation(fields: [orderId], references: [id], onDelete: Restrict)

  @@map("disputes")
}

model Review {
  id          String      @id @default(uuid()) @db.Uuid
  orderId     String      @unique @map("order_id") @db.Uuid
  rating      Int         // Strictly 1 to 5
  body        String?     @db.VarChar(500)
  createdAt   DateTime     @default(now()) @map("created_at") @db.Timestamptz

  order      Order      @relation(fields: [orderId], references: [id], onDelete: Cascade)

  @@map("reviews")
}

// -----
// MESSAGING & NOTIFICATIONS
// -----

model Message {
  id          String      @id @default(uuid()) @db.Uuid
  orderId     String      @map("order_id") @db.Uuid
  senderId    String      @map("sender_id") @db.Uuid
  body        String      @db.Text
  attachmentKey String?   @map("attachment_key") @db.VarChar(512)
  readAt      DateTime?   @map("read_at") @db.Timestamptz
  createdAt   DateTime     @default(now()) @map("created_at") @db.Timestamptz

  order      Order      @relation(fields: [orderId], references: [id], onDelete: Cascade)
  sender      User       @relation(fields: [senderId], references: [id], onDelete: Restrict)

  @@index([orderId, createdAt])
  @@map("messages")
}

model Notification {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @map("user_id") @db.Uuid
  type        NotificationType
  orderId     String?     @map("order_id") @db.Uuid
  message     String      @db.VarChar(255)
  readAt      DateTime?   @map("read_at") @db.Timestamptz
  createdAt   DateTime     @default(now()) @map("created_at") @db.Timestamptz

  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId, readAt])
  @@map("notifications")
}

model NotificationPref {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @unique @map("user_id") @db.Uuid
  orderEmail  Boolean      @default(true) @map("order_email")
  inboxEmail  Boolean      @default(true) @map("inbox_email")
  promoEmail  Boolean      @default(false) @map("promo_email")

  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@map("notification_prefs")
}

// -----
// DOUBLE-ENTRY FINANCIAL LEDGER
// -----

model LedgerAccount {

```

```

id          String          @id @default(uuid()) @db.Uuid
ownerUserId String?         @map("owner_user_id") @db.Uuid
kind        LedgerAccountKind
createdAt   DateTime        @default(now()) @map("created_at") @db.Timestamptz

user        User?           @relation(fields: [ownerUserId], references: [id], onDelete: Restrict)
entries     LedgerEntry[]

@@unique([ownerUserId, kind])
@@map("ledger_accounts")
}

model LedgerEntry {
  id          String          @id @default(uuid()) @db.Uuid
  txnId       String          @map("txn_id") @db.Uuid
  accountId   String          @map("account_id") @db.Uuid
  orderId     String?         @map("order_id") @db.Uuid
  amount      Int             // Integer cents (debit negative, credit positive)
  entryType   LedgerEntryType @map("entry_type")
  createdAt   DateTime        @default(now()) @map("created_at") @db.Timestamptz

  account     LedgerAccount    @relation(fields: [accountId], references: [id], onDelete: Restrict)
  order       Order?           @relation(fields: [orderId], references: [id], onDelete: Restrict)

  @@index([txnId])
  @@index([accountId])
  @@index([orderId, entryType])
  @@map("ledger_entries")
}

```

Appendix C: Docker Compose (Local Development)

```

version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: microgig_postgres
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres_secure_pass
      POSTGRES_DB: microgig
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./scripts/init-ledger-trigger.sql:/docker-entrypoint-initdb.d/init-ledger-trigger.sql

  redis:
    image: redis:7-alpine
    container_name: microgig_redis
    restart: always
    command: redis-server --requirepass redis_secure_pass
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  minio:
    image: minio/minio:latest
    container_name: microgig_minio
    restart: always
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - minio_data:/data

volumes:
  postgres_data:
  redis_data:
  minio_data:

```

Appendix D: Testing & Verification Strategy

Test Layer	Target Scope	Execution Framework	Pass / Exit Criteria
Unit Tests	State machine transitions, optimistic lock retry, fee rounding, Zod validation	Vitest	95%+ coverage on <code>src/modules/orders</code> and <code>src/modules/ledger</code> .
Invariant Tests	Ledger zero-sum assertion, balance boundary violations, non-negative checks	Vitest + PostgreSQL	100% of tested transactions satisfy $\sum \text{ext}\{\text{amount}\} = 0\$$. Unbalanced transactions trigger SQL rollback.
Integration Tests	REST API flows (Register \$ o\$ Onboard \$ o\$ Create Gig \$ o\$ Order \$ o\$ Deliver \$ o\$ Accept)	Supertest + Next.js Test Server	HTTP 200/201 on all standard paths; proper 401/402/403/409/422 responses on error cases.
IDOR Security Tests	Cross-tenant access to private order workspaces, messages, and deliverables	Automated HTTP Suite	100% rejection with HTTP 403 Forbidden when unauthorized user attempts access.
Concurrency Tests	Concurrent "Accept" vs. "Auto-Complete" vs. "Revision" requests	k6 / Artillery	Optimistic locking prevents race conditions; exactly 1 transition succeeds; loser receives 409 Conflict.
End-to-End Tests	Browser journeys: Catalog search, 5-step wizard, workspace delivery, and Wallet	Playwright	Flawless execution across Chromium, Firefox, and Mobile Safari emulation.

Appendix E: Production Deployment Checklist

- [] **Database Setup:** Managed PostgreSQL (Supabase/Neon) provisioned with connection pooler enabled.
- [] **Migrations & Triggers:** Prisma migrations applied and `trg_assert_ledger_zero_sum` trigger verified active in database.
- [] **Redis Instance:** Upstash / Railway Redis instance online with persistence enabled.
- [] **Object Storage:** S3/R2 buckets created with CORS configuration allowing client direct `PUT`.
- [] **Bucket Policies:** Public read configured exclusively for `microgig-showcase-public`; private access strictly enforced for `microgig-deliverables-private`.
- [] **Background Workers:** Dedicated worker process (`npm run worker`) running in production (handling BullMQ delayed jobs, 10m reconciler, and 3-day clearing).
- [] **Secrets & Keys:** JWT secrets, cookie secrets, and S3 credentials rotated from development values.
- [] **Security Headers:** HSTS, CSP, X-Frame-Options, X-Content-Type-Options headers active in `next.config.mjs`.
- [] **Rate Limiting:** Sliding window rate limits verified active on authentication and upload endpoints.
- [] **Nightly Audit Job:** Scheduled SQL assertion job enabled to verify ledger balance integrity daily at 00:00 UTC.
- [] **Health Check Verified:** `GET /api/v1/health` reports status `OK` for database, redis, and cloud storage connections.

Document Status: Approved v1.1. This document serves as the binding functional, architectural, and security contract for the Micro-Gig engineering team. All code submissions must adhere strictly to these specifications.